

DOSSIER PROFESSIONNEL (DP)

Nom de naissance ▶ Lecomte
Nom d'usage ▶ Lecomte
Prénom ▶ Anne-Laure
Adresse ▶

Titre professionnel visé

Concepteur Développeur d'Application

MODALITE D'ACCES :

- ☒ Parcours de formation
- ☐ Validation des Acquis de l'Expérience (VAE)

Présentation du dossier

Le dossier professionnel (DP) constitue un élément du système de validation du titre professionnel. **Ce titre est délivré par le Ministère chargé de l'emploi.**

Le DP appartient au candidat. Il le conserve, l'actualise durant son parcours et le présente **obligatoirement à chaque session d'examen.**

Pour rédiger le DP, le candidat peut être aidé par un formateur ou par un accompagnateur VAE.
Il est consulté par le jury au moment de la session d'examen.

Pour prendre sa décision, le jury dispose :

1. des résultats de la mise en situation professionnelle complétés, éventuellement, du questionnaire professionnel ou de l'entretien professionnel ou de l'entretien technique ou du questionnement à partir de productions.
2. du **Dossier Professionnel** (DP) dans lequel le candidat a consigné les preuves de sa pratique professionnelle
3. des résultats des évaluations passées en cours de formation lorsque le candidat évalué est issu d'un parcours de formation
4. de l'entretien final (dans le cadre de la session titre).

[Arrêté du 22 décembre 2015, relatif aux conditions de délivrance des titres professionnels du ministère chargé de l'Emploi]

Ce dossier comporte :

- ▶ pour chaque activité-type du titre visé, un à trois exemples de pratique professionnelle ;
- ▶ un tableau à renseigner si le candidat souhaite porter à la connaissance du jury la détention d'un titre, d'un diplôme, d'un certificat de qualification professionnelle (CQP) ou des attestations de formation ;
- ▶ une déclaration sur l'honneur à compléter et à signer ;
- ▶ des documents illustrant la pratique professionnelle du candidat (facultatif)
- ▶ des annexes, si nécessaire.

Pour compléter ce dossier, le candidat dispose d'un site web en accès libre sur le site.



<http://travail-emploi.gouv.fr/titres-professionnels>

Sommaire

Exemples de pratique professionnelle

Développer une application sécurisée

p. 5

- ▶ Installer et configurer son environnement de travail p. 5
- ▶ Développer des interfaces utilisateur p. 17
- ▶ Développer des composants métier p. 37
- ▶ Contribuer à la gestion d'un projet informatique p. 51

Titres, diplômes, CQP, attestations de formation *(facultatif)*

p. _____

Déclaration sur l'honneur

p. _____

Documents illustrant la pratique professionnelle *(facultatif)*

p. _____

Annexes *(Si le RC le prévoit)*

p. _____

EXEMPLES DE PRATIQUE PROFESSIONNELLE

Activité-type 1 Développer une application sécurisée

CP 1 ▶ Installer et configurer son environnement de travail en fonction du projet

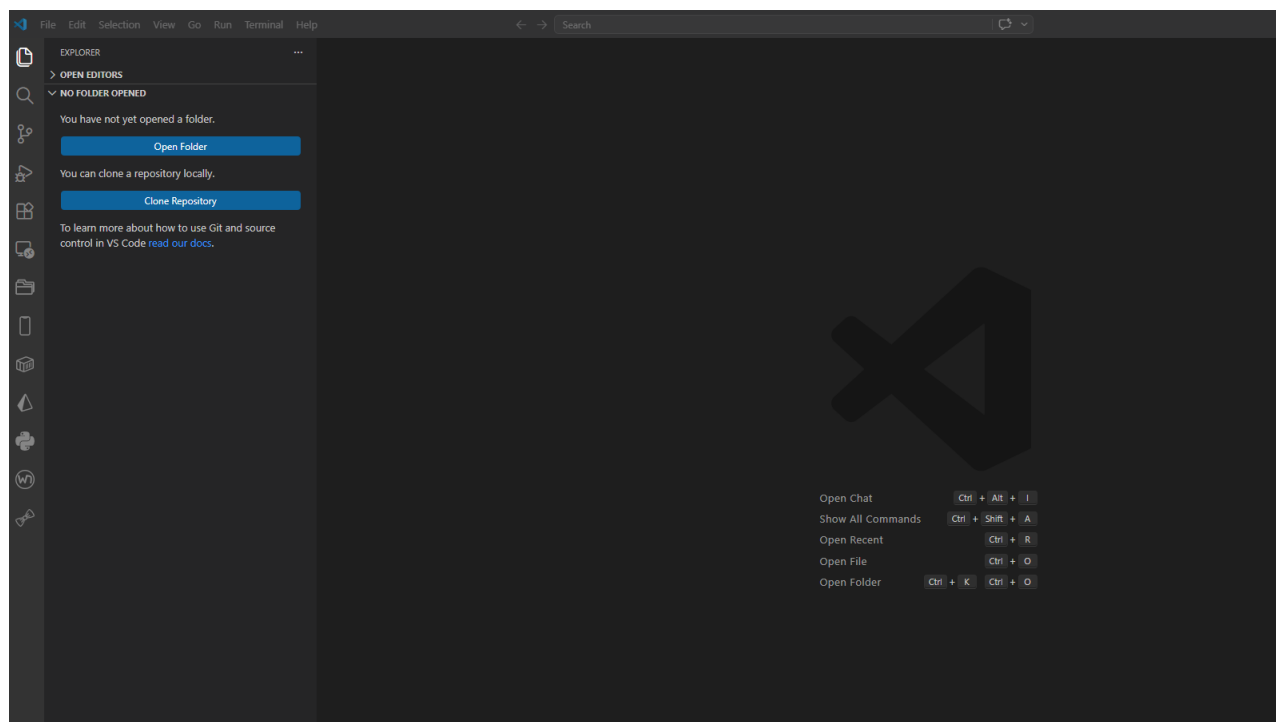
1. Décrivez les tâches ou opérations que vous avez effectuées, et dans quelles conditions :

Dans le cadre de la mise en place de mon environnement de développement, j'ai réalisé plusieurs opérations :

Installation d'outils de développement :

Visual Studio Code avec extensions :

J'ai installé Visual Studio Code comme éditeur principal et configuré un ensemble d'extensions pour optimiser ma productivité et la qualité de code.



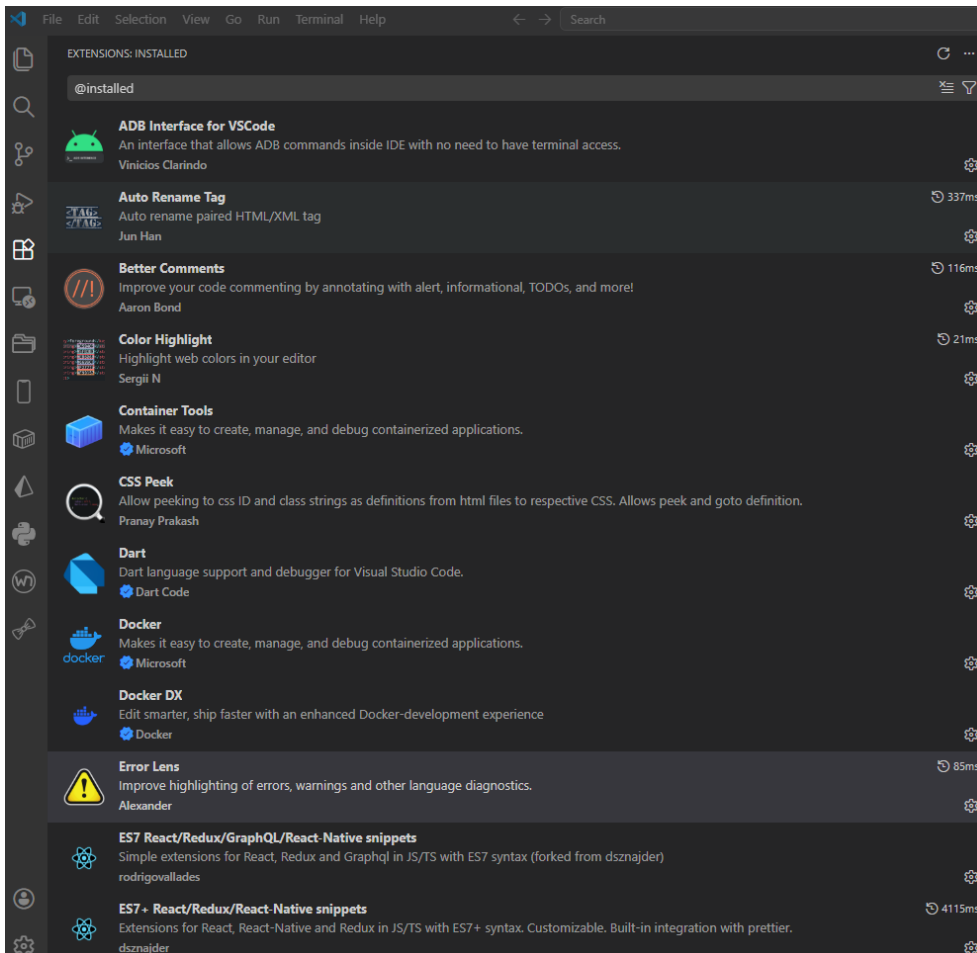
Interface Visual Studio Code

Les extensions installées incluent notamment :

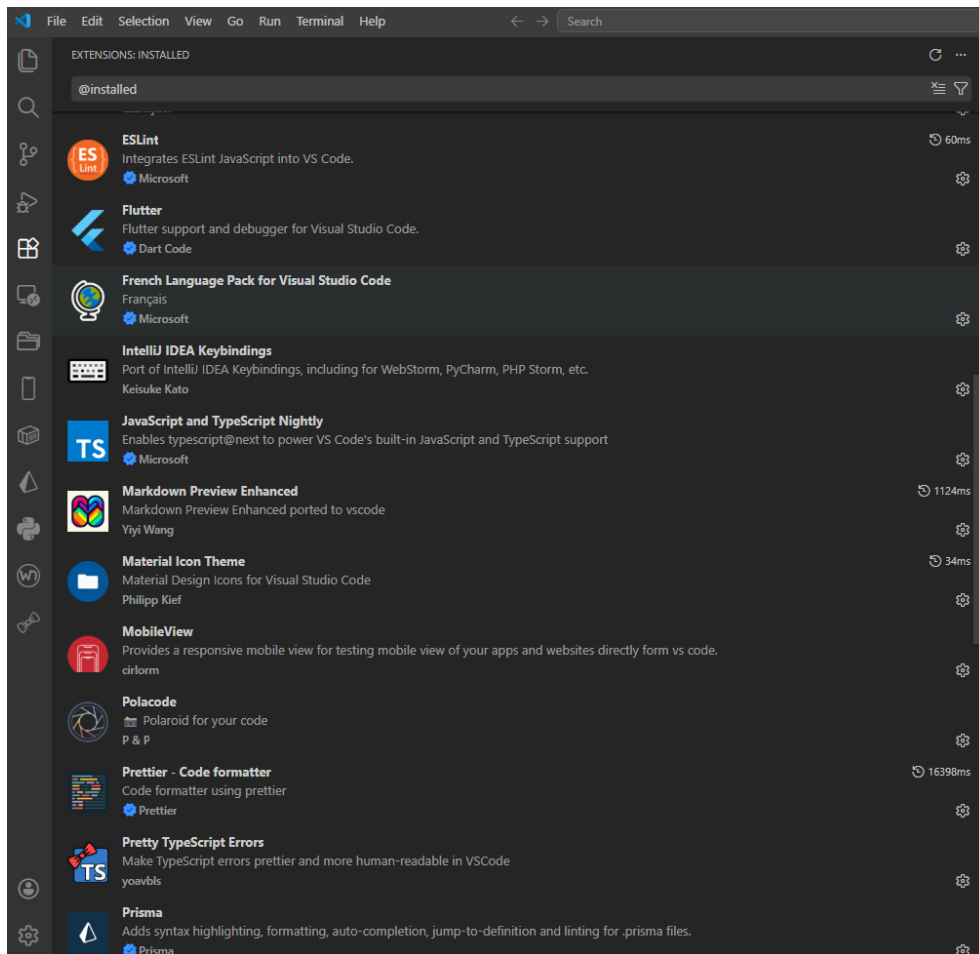
- **ADB Interface for VSCode** : pour interagir avec les appareils Android directement depuis VSCode
- **Better Comments** : pour améliorer la lisibilité du code avec des commentaires colorés et catégorisés
- **Dart** : extension officielle pour le langage Dart offrant l'autocomplétion, le débogage et l'analyse statique du code Dart

DOSSIER PROFESSIONNEL (DP)

- **ESLint** : pour analyser les erreurs de syntaxe et les mauvaises pratiques en JavaScript/TypeScript
- **Flutter** : extension officielle pour Flutter incluant des outils pour le développement mobile
- **Markdown Preview Enhanced** : pour prévisualiser et faciliter l'édition des fichiers Markdown
- **MobileView** : pour simuler l'affichage mobile dans VSCode et tester le responsive design
- **Prettier** : pour le formatage automatique du code (indentation, sauts de ligne...)



Liste des extensions installées (partie 1)



Liste des extensions installées (partie 2)

Installation de Node.js avec nvm :

J'ai fait le choix d'installer Node.js avec nvm (Node Version Manager) car cela permet d'installer et de gérer des versions différentes de Node.js. Je peux donc basculer facilement entre les versions selon les projets.

J'ai donc installé nvm et ensuite installé la version de Node.js dont j'avais besoin.

```
Windows PowerShell
PS C:\Users\alecomte> nvm list

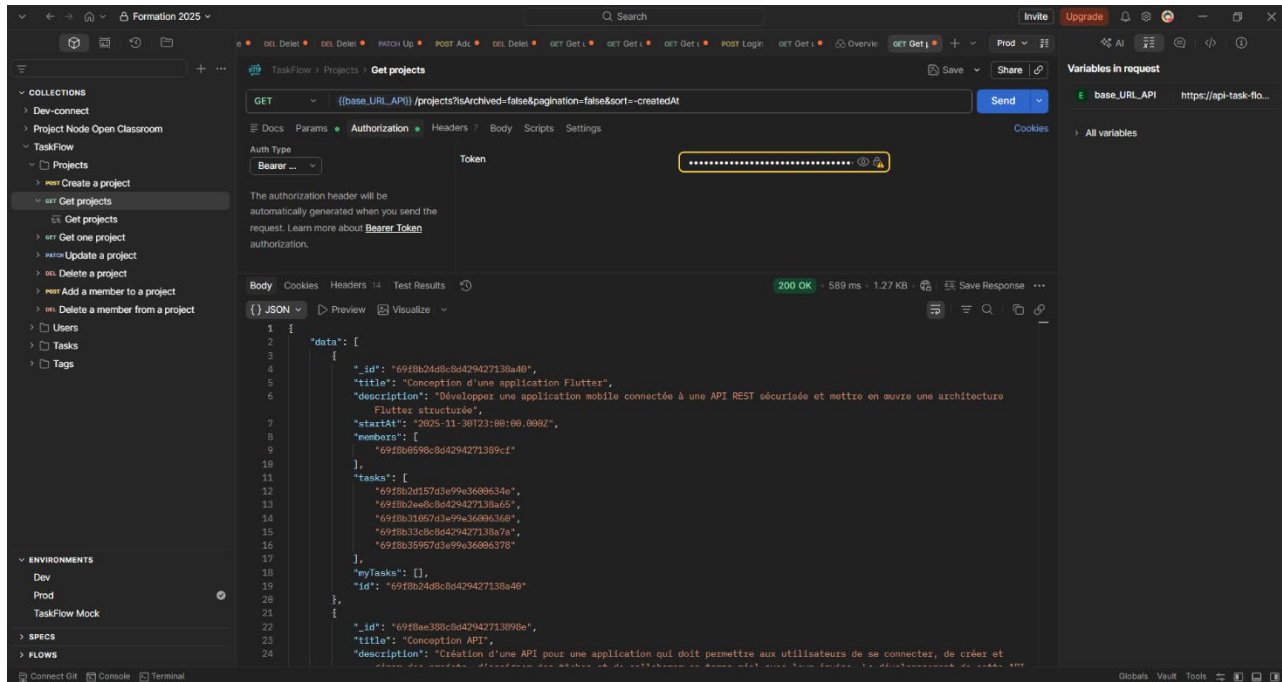
 24.10.0
 20.20.2
 * 20.9.0 (Currently using 64-bit executable)
 12.22.12
 12.18.3
PS C:\Users\alecomte> nvm install 24.15.0
Downloading node.js version 24.15.0 (64-bit)...
Extracting node and npm...
Complete
Installation complete.
If you want to use this version, type:

nvm use 24.15.0
PS C:\Users\alecomte> nvm use 24.15.0
Now using node v24.15.0 (64-bit)
```

Installation de la version 24.15.0 de Node.js via nvm

Postman pour tester les requêtes :

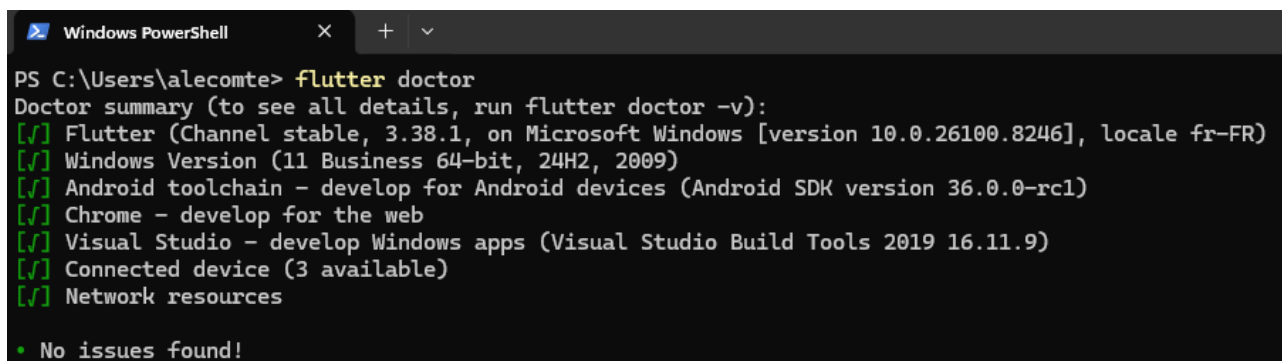
J'ai installé Postman pour pouvoir facilement tester des requêtes en créant des Collections par projet et en définissant différents environnements.



Interface Postman avec une requête GET /projects exécutée dans un environnement de production

Installation Flutter :

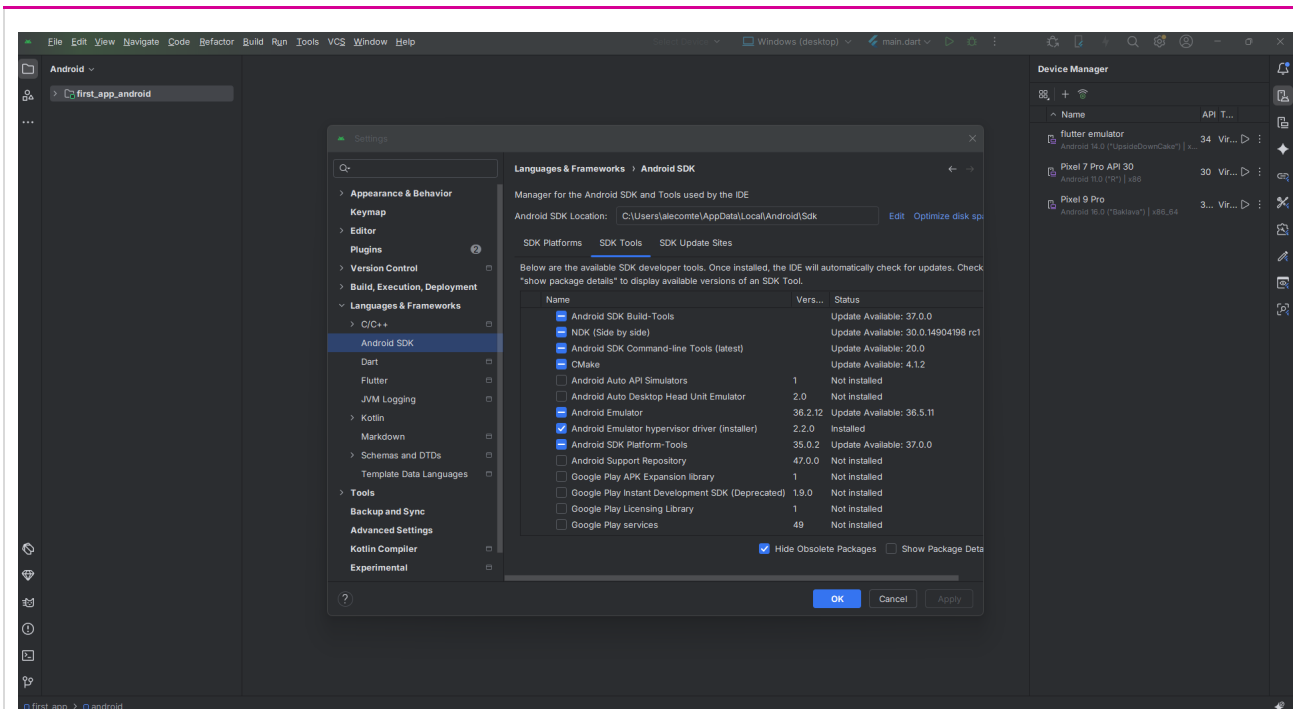
J'ai installé Flutter et le Flutter SDK via Visual Studio Code. J'ai vérifié ensuite la bonne installation avec flutter doctor.



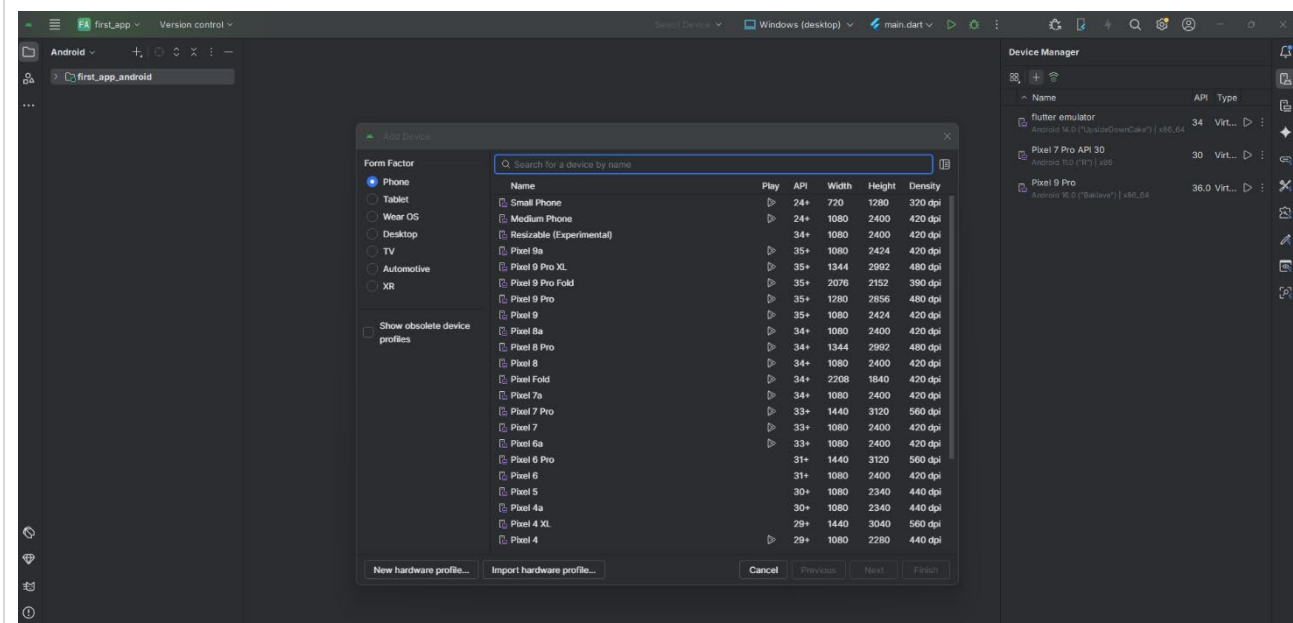
Résultat de flutter doctor – aucun problème trouvé

J'ai utilisé Android Studio pour m'assurer que tous les composants étaient bien installés et pour ajouter des émulateurs.

DOSSIER PROFESSIONNEL (DP)



Interface Android Studio – SDK Manager

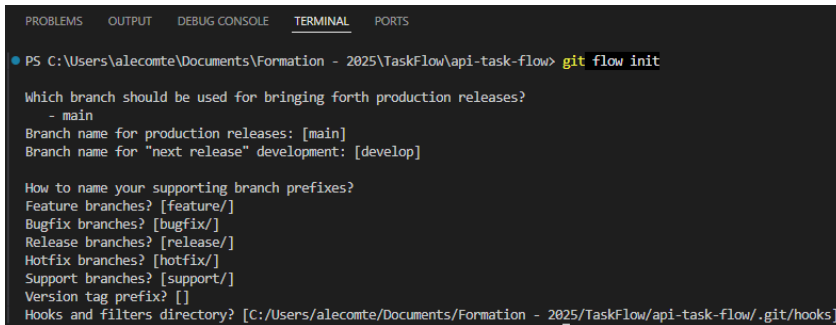


Interface Android Studio – Configuration d'un émulateur avec choix du modèle

Versioning :

J'ai installé Git et j'ai configuré ma connexion avec GitHub pour avoir un suivi de versions de mes projets et de pouvoir partager le code. J'ai créé des dépôts distants pour mes différents projets me permettant de sauvegarder régulièrement mon travail. J'ai également installé Git Flow pour mieux organiser le développement en séparant les features, releases et corrections de bug dans des branches dédiées.

Cela facilite la gestion des versions pour avoir un historique propre et des déploiements contrôlés. De plus, dans le cadre d'un travail en équipe cela facilite la collaboration grâce à des règles de branchement claires.



```
PS C:\Users\alecomte\Documents\Formation - 2025\TaskFlow\api-task-flow> git flow init

Which branch should be used for bringing forth production releases?
- main
Branch name for production releases: [main]
Branch name for "next release" development: [develop]

How to name your supporting branch prefixes?
Feature branches? [feature/]
Bugfix branches? [bugfix/]
Release branches? [release/]
Hotfix branches? [hotfix/]
Support branches? [support/]
Version tag prefix? []
Hooks and filters directory? [C:/Users/alecomte/Documents/Formation - 2025/TaskFlow/api-task-flow/.git/hooks]
```

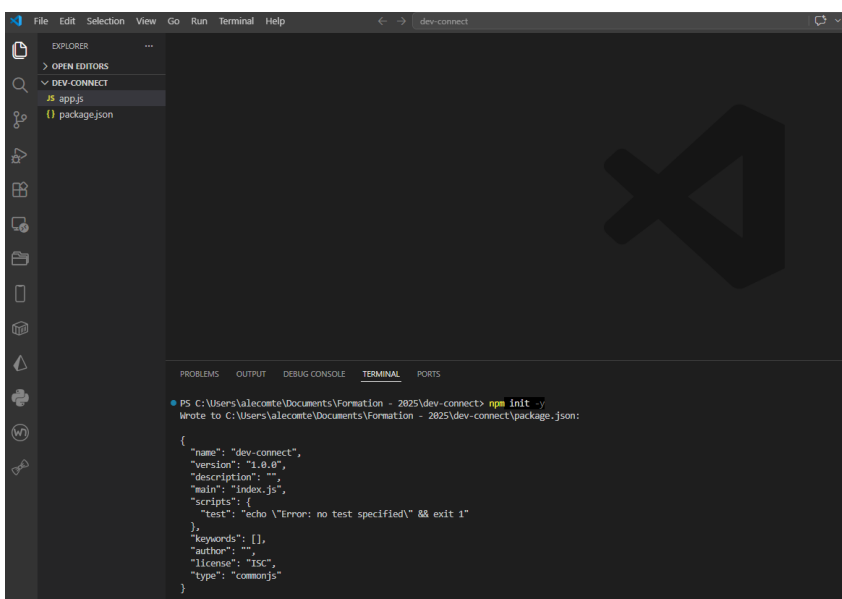
Initialisation de Git Flow sur un projet

Configuration des projets de développement :

Dans le cadre de mes projets, j'ai configuré différents environnements de développement pour répondre à des besoins variés. Voici deux exemples représentatifs : un backend en Node.js avec Express et MongoDB pour une API REST, et un projet mobile en Flutter pour une application cross-platform. Ces configurations illustrent ma capacité à adapter mon environnement technique en fonction des technologies utilisées.

Configuration d'un projet Node.js avec Express et MongoDB :

Afin de mettre en place un projet Node.js, j'ai créé un dossier où j'ai initié le projet et installé les dépendances nécessaires avec npm.



```
PS C:\Users\alecomte\Documents\Formation - 2025\dev-connect> npm init -y
Wrote to C:\Users\alecomte\Documents\Formation - 2025\dev-connect\package.json:

{
  "name": "dev-connect",
  "version": "1.0.0",
  "description": "",
  "main": "index.js",
  "scripts": {
    "test": "echo \"Error: no test specified\" && exit 1"
  },
  "keywords": [],
  "author": "",
  "license": "ISC",
  "type": "commonjs"
}
```

Initialisation du projet avec la commande npm init -y

```
PS C:\Users\alecomte\Documents\Formation - 2025\dev-connect> npm install express mongoose dotenv cors

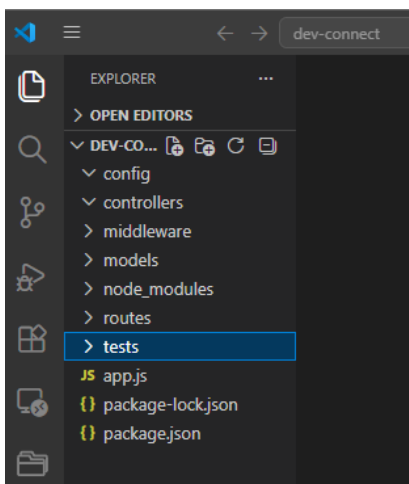
added 85 packages, and audited 86 packages in 25s

25 packages are looking for funding
  run `npm fund` for details

found 0 vulnerabilities
```

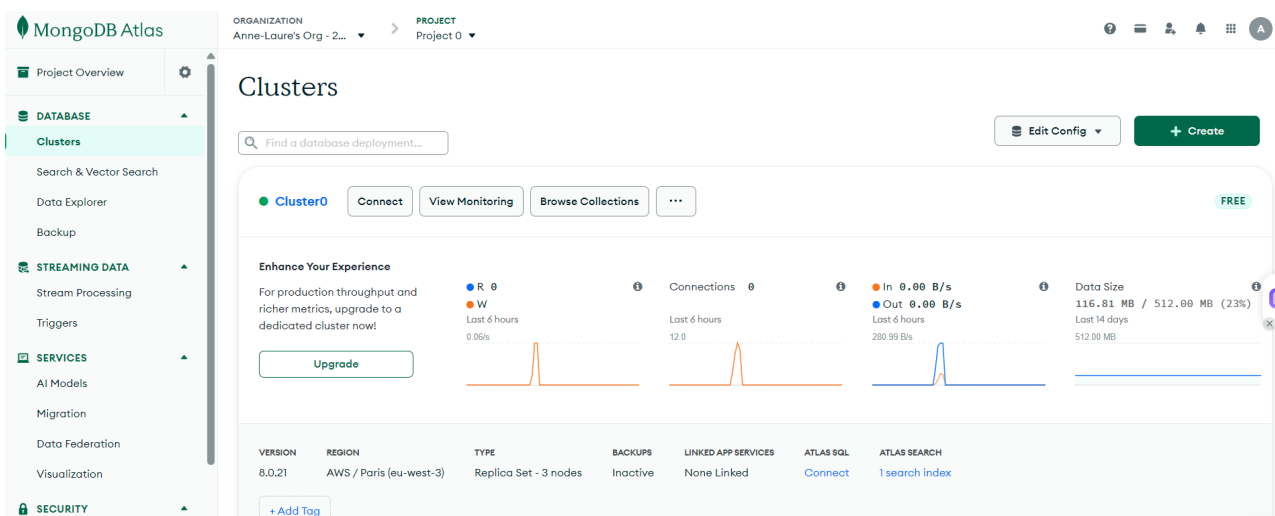
Installation des dépendances

J'ai ensuite créé les dossiers nécessaires à une bonne structure de projet et créé le fichier app.js qui sera le point d'entrée de l'application.



Structure des fichiers d'un projet Node.js

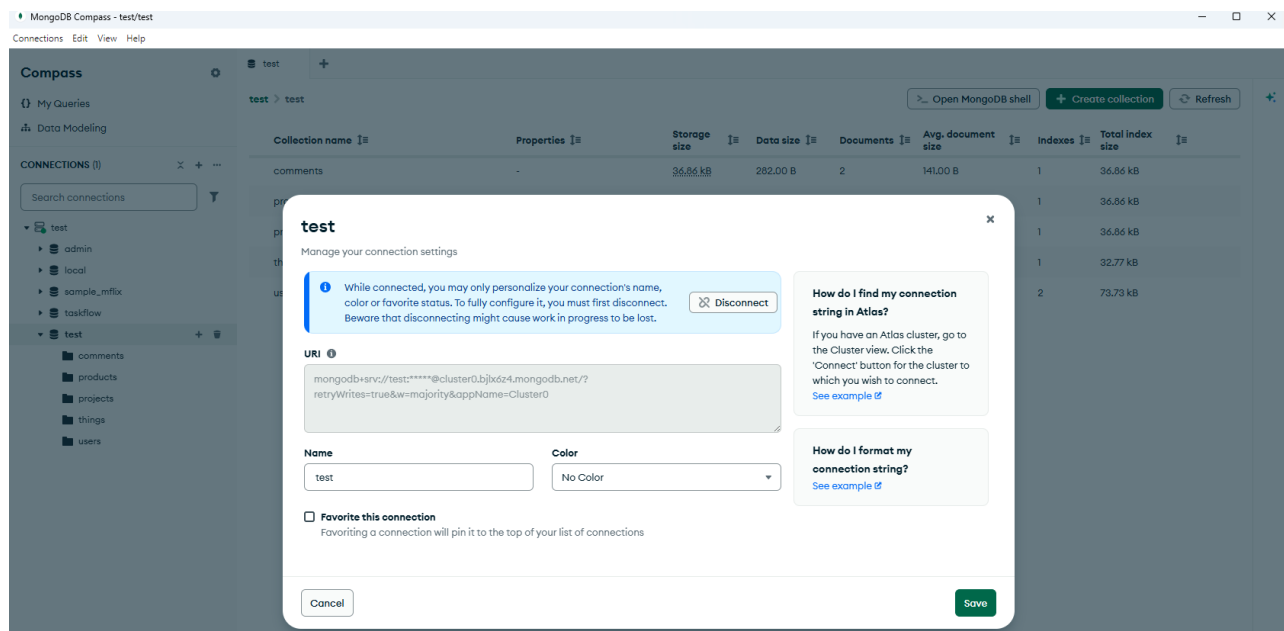
Pour établir la connexion à la base de données MongoDB, j'ai d'abord ajouté un Cluster dans MongoDB Atlas qui est une solution cloud pour héberger la base.



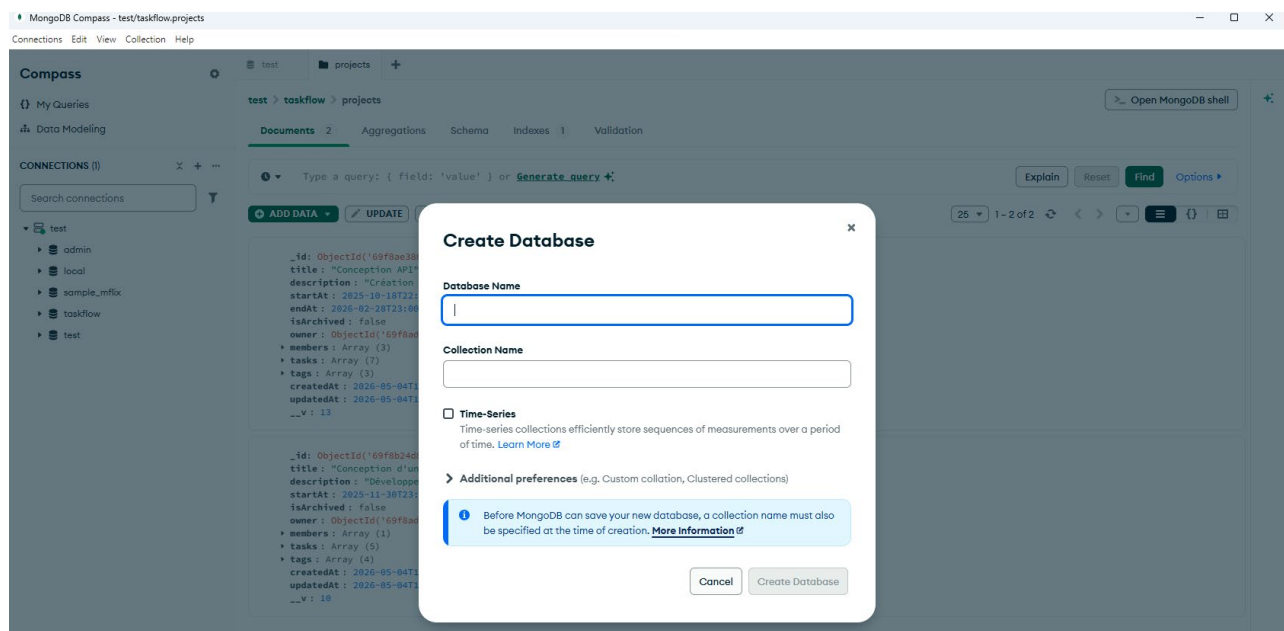
Interface MongoDB Atlas avec un Cluster configuré

DOSSIER PROFESSIONNEL (DP)

J'ai installé MongoDB Compass pour visualiser facilement les données de la base et pouvoir interagir avec.



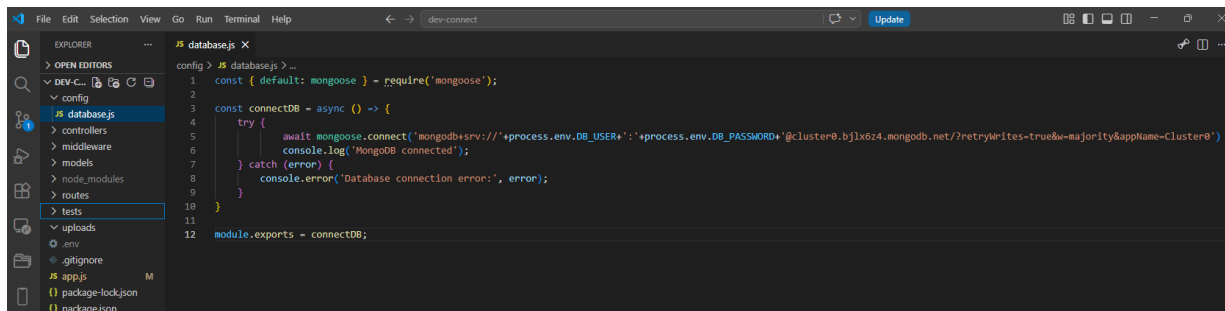
Interface MongoDB Compass de connexion au Cluster



Interface MongoDB Compass de création d'une base de données

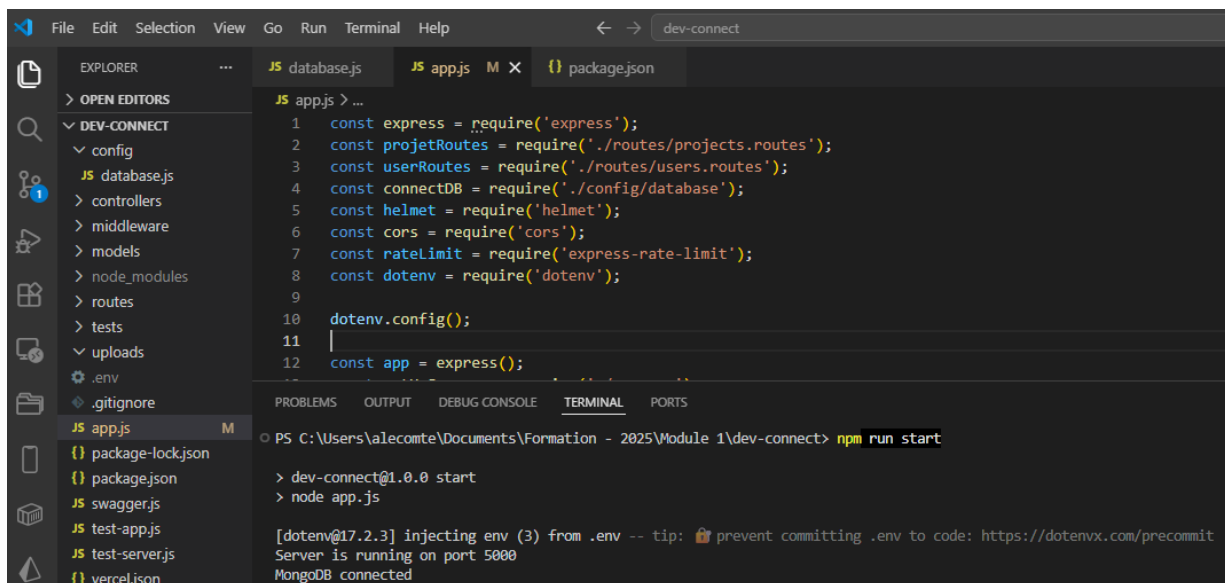
L'URL fournit par MongoDB Atlas est utilisé via Mongoose pour la connexion à la base et les identifiants sont placés dans le fichier .env pour les sécuriser.

DOSSIER PROFESSIONNEL (DP)



Fichier de configuration de la base de données

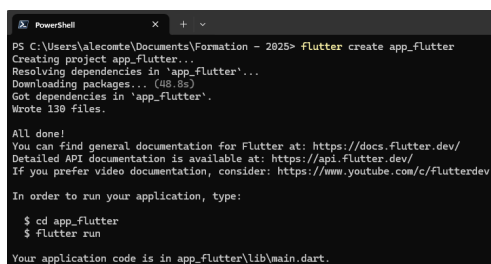
J'ai ensuite configuré le serveur Express dans le fichier app.js et ajouté un script de démarrage dans le package.json.



Exécution du script de démarrage avec le serveur Express sur le port 5000 et la connexion à MongoDB

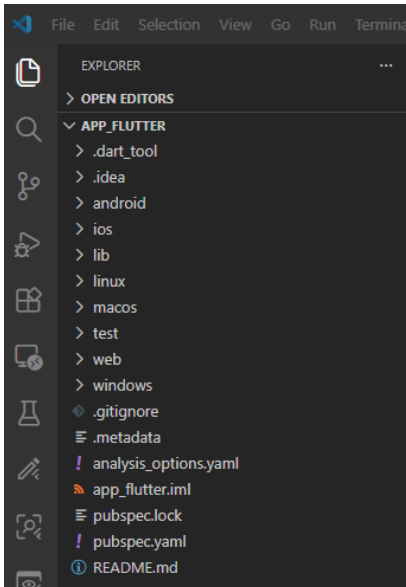
Configuration d'un projet Flutter :

Afin de créer un nouveau projet Flutter, j'ai lancé la commande flutter create suivi du nom du projet.



Exécution de la commande flutter create

Cela génère la structure du projet.



Structure des fichiers d'un projet Flutter

J'ai ensuite créé un émulateur et lancer la commande flutter run pour tester le bon fonctionnement du projet.

```
PS C:\Users\alecomte\Documents\Formation - 2025\app_flutter> flutter devices
Found 4 connected devices:
  sdk gphone64 x86 64 (mobile) • emulator-5554 • android-x64 • Android 14 (API 34) (emulator)
  Windows (desktop)          • windows      • windows-x64 • Microsoft Windows [version 10.0.26100.8246]
  Chrome (web)               • chrome       • web-javascript • Google Chrome 137.0.7151.120
  Edge (web)                 • edge         • web-javascript • Microsoft Edge 147.0.3912.98

Run "flutter emulators" to list and start any available device emulators.
```

Commande flutter devices pour vérifier que l'émulateur créé est bien reconnu

```
PS C:\Users\alecomte\Documents\Formation - 2025\app_flutter> flutter run
Launching lib/main.dart on sdk gphone64 x86 64 in debug mode...
Running Gradle task 'assembleDebug'... 312,8s
✓ Built build/app/outputs/flutter-apk/app-debug.apk
Installing build/app/outputs/flutter-apk/app-debug.apk... 17,5s
D/FlutterJNI( 8672): Beginning load of flutter...
D/FlutterJNI( 8672): Flutter (null) was loaded normally!
I/Flutter ( 8672): [IMPORTANT:flutter/shell/platform/android/android_context_gl_impeller.cc(104)] Using the Impeller rendering backend (OpenGL).
I/Choreographer( 8672): Skipped 487 frames! The application may be doing too much work on its main thread.
I/6ralloc4( 8672): mapper 4.x is not supported
W/OpenGLRenderer( 8672): Failed to choose config with EGL_SWAP_BEHAVIOR_PRESERVED, retrying without...
W/OpenGLRenderer( 8672): Failed to initialize 101010-2 format, error = EGL_SUCCESS
E/OpenGLRenderer( 8672): Unable to match the desired swap behavior.
I/Choreographer( 8672): Skipped 124 frames! The application may be doing too much work on its main thread.
Syncing files to device sdk gphone64 x86 64... 2 499ms

Flutter run key commands.
r Hot reload.
R Hot restart.
h List all available interactive commands.
d Detach (terminate "flutter run" but leave application running).
c Clear the screen
q Quit (terminate the application on the device).

A Dart VM Service on sdk gphone64 x86 64 is available at: http://127.0.0.1:59870/Ah7eRuTEQ3M=/
The Flutter DevTools debugger and profiler on sdk gphone64 x86 64 is available at: http://127.0.0.1:59870/Ah7eRuTEQ3M=/devtools/?uri=ws://127.0.0.1:59870/Ah7eRuTEQ3M=/ws
D/ProfileInstaller( 8672): Installing profile for com.example.app_flutter
```

Exécution de la commande flutter run pour lancer l'application

2. Précisez les moyens utilisés :

Environnement matériel :

- Ordinateur avec processeur performant
- Système d'exploitation Windows 11

Logiciels et outils :

- Visual Studio Code : Éditeur de code avec des extensions
- Android Studio : IDE incluant émulateurs
- Git Flow : Workflow Git structuré pour gérer les branches
- GitHub : Plateforme de gestion de dépôts distants en ligne pour le versioning
- MongoDB Atlas : Service cloud de base de données NoSQL pour héberger et gérer des bases de données MongoDB en ligne
- MongoDB Compass : Interface graphique pour visualiser, requêter et gérer les données MongoDB
- Postman : Outil permettant d'envoyer des requêtes http

DOSSIER PROFESSIONNEL (DP)

3. Avec qui avez-vous travaillé ?

J'ai travaillé en autonomie sur la configuration de mon environnement de développement en suivant les consignes de formation et la documentation disponible en ligne sur les différents langages, frameworks, bibliothèques, logiciels et outils.

J'ai également bénéficié de l'accompagnement d'un formateur avec qui j'ai échangé sur les bonnes pratiques et qui a validé ma configuration. Ces échanges ont eu lieu via des sessions de formations en distanciel.

4. Contexte

Nom de l'entreprise, organisme ou association ► **Global Digital University**

Chantier, atelier, service ► Initialiser le projet Node.js et configurer l'environnement

Période d'exercice ► Du 13/10/2025 au 20/10/2025

5. Informations complémentaires (facultatif)

Activité-type 1 Développer une application sécurisée

CP 2 ► Développer des interfaces utilisateur

1. Décrivez les tâches ou opérations que vous avez effectuées, et dans quelles conditions :

Dans le cadre de ma formation, j'ai conçu une application mobile en Flutter permettant d'afficher les prévisions météorologiques en temps réel. Ce projet illustre ma maîtrise du développement d'interfaces utilisateur en intégrant des fonctionnalités clés comme la géolocalisation, la consommation d'une API externe (OpenWeather), et la persistance de données locales via SharedPreferences.

Objectif :

Créer une application affichant les prévisions météorologiques incluant une géolocalisation de l'utilisateur

Public cible :

Tout utilisateur de smartphone

Fonctionnalités principales :

- Géolocalisation automatique pour afficher la météo de la position actuelle
- Recherche de villes pour afficher la météo d'une ville spécifique
- Sauvegarde de villes recherchées pour un accès rapide ultérieur
- Affichage des prévisions (température, conditions, icônes météo)

Technologies :

- Flutter (Dart) avec les packages : http, location, geocoding, shared_preferences , permission_handler
- API externe : OpenWeather

Architecture de l'application :

Pour structurer mon application, j'ai adopté une architecture qui sépare les responsabilités entre les services, les modèles de données, les vues et les widgets. Cette approche permet une meilleure maintenabilité et une réutilisabilité du code.

DOSSIER PROFESSIONNEL (DP)

```
lib/
├── models/
│   ├── api_response.dart
│   ├── geo_position.dart
│   ├── grouped_weather.dart
│   └── location_result.dart
├── services/
│   ├── api_service.dart
│   ├── data_converter.dart
│   ├── data_service.dart
│   └── location_service.dart
├── views/
│   └── home_view.dart
├── widgets/
│   ├── add_city_form.dart
│   ├── current_weather.dart
│   ├── daily_title.dart
│   ├── forecast_data.dart
│   ├── my_drawer.dart
│   └── no_data_view.dart
└── main.dart
```

Fichier	Description
models/api_response.dart	Modélisation des données météo retournées par l'API
models/geo_position.dart	Modélisation des coordonnées géographiques
models/grouped_weather.dart	Modélisation des données météo à afficher
models/location_result.dart	Modélisation des résultats de géolocalisation (succès/échec)
services/api_service.dart	Appels à l'API OpenWeather
services/data_converter.dart	Convertir les données de l'API en objet Dart
services/data_service.dart	Gestion du stockage local (SharedPreferences)
services/location_service.dart	Géolocalisation et conversion coordonnées/ville
views/home_view.dart	Vue principale de l'application
widgets/add_city_form.dart	Formulaire pour ajouter une ville
widgets/current_weather.dart	Widget pour afficher la météo du jour
widgets/daily_title.dart	Widget pour afficher les données météo d'une journée à venir
widgets/forecast_data.dart	Widget pour afficher toutes les prévisions
widgets/my_drawer.dart	Menu latéral pour les villes sauvegardées
widgets/no_data_view.dart	Widget affiché en cas de données manquantes ou non disponibles
main.dart	Point d'entrée de l'application

1 - Intégration de l'API OpenWeather :

L'application utilise l'API OpenWeather pour récupérer les prévisions météorologiques. Cette intégration a été réalisée en plusieurs étapes :

Construction de l'URL et appel http :

Dans le fichier **api_service.dart**, j'ai implémenté une fonction `prepareQuery` pour construire dynamiquement l'URL de requête en fonction des coordonnées géographiques (latitude et longitude). L'appel HTTP est ensuite effectué avec le package `http`.

```
class ApiService {}  
String baseUrl = "https://api.openweathermap.org/data/2.5/forecast";  
String lat = "lat=";  
String lon = "lon=";  
String appid = "appid=";  
String lang = "lang=fr";  
String units = "units=metric";  
  
String prepareQuery(GeoPosition geoPosition) {  
  final geoLat = geoPosition.latitude;  
  final geoLon = geoPosition.longitude;  
  return "$baseUrl?$lat$geoLat&$lon$geoLon&$units&$lang&$appid$API_KEY";  
}  
  
Future<APIResponse> callApi(GeoPosition position) async {  
  final queryString = prepareQuery(position);  
  final uri = Uri.parse(queryString);  
  final call = await get(uri);  
  Map<String, dynamic> map = json.decode(call.body);  
  return APIResponse.fromJson(map);  
}
```

Fichier `api_service.dart` qui gère les appels vers l'API OpenWeather

- L'URL est construite dynamiquement à partir de position passée en paramètre.
- Les paramètres `units=metric` et `lang=fr` garantissent que les données sont retournées en degrés Celsius et en français.
- Le parsing du JSON est effectué via le constructeur `APIResponse.fromJson`, qui convertit les données brutes en objets Dart structurés.

Modélisation des données :

Les données retournées par l'API OpenWeather sont complexes et imbriquées. J'ai donc créé une hiérarchie de classes dans `api_response.dart` pour les modéliser.

```
class APIResponse {
  String cod;
  int message;
  int cnt;
  List<Forecast> list;

  APIResponse(this.cod, this.message, this.list, this.cnt);

  APIResponse.fromJson(Map<String, dynamic> map) :
    cod = map["cod"],
    message = map["message"],
    cnt = map["cnt"],
    list = DataConverter().listMappable(map["list"]).map((e) => Forecast.fromJson(e)).toList();
}

class Forecast {
  int dt;
  Main main;
  List<Weather> weather;
  Wind wind;
  Clouds clouds;
  int visibility;
  String dt_txt;

  Forecast(
    this.dt,
    this.main,
    this.weather,
    this.wind,
    this.clouds,
    this.visibility,
    this.dt_txt
  );

  Forecast.fromJson(Map<String, dynamic> map):
    dt = map["dt"],
    main = Main.fromJson(map["main"]),
    weather = DataConverter().listMappable(map["weather"]).map((e) => Weather.fromJson(e)).toList(),
    wind = Wind.fromJson(map["wind"]),
    clouds = Clouds.fromJson(map["clouds"]),
    visibility = map["visibility"],
    dt_txt = map["dt_txt"];
}
```

Extrait du fichier `api_response.dart` qui modélise les données météo retournées par l'API

- La classe `APIResponse` représente la réponse globale de l'API, contenant une liste de `Forecast` (prévisions).
- Chaque `Forecast` contient des sous-objets comme `Main` (les informations principales comme la température), `Weather` (les conditions) ...
- Le parsing est automatisé grâce aux constructeurs `fromJson`, ce qui simplifie l'intégration et l'utilisation des données.

2 - Géolocalisation et conversion des coordonnées :

La géolocalisation est une fonctionnalité centrale de l'application. Elle permet d'afficher automatiquement la météo de la position actuelle de l'utilisateur.

Récupération de la position utilisateur :

Dans `location_service.dart`, j'ai utilisé le package `location` pour récupérer les coordonnées géographiques (latitude et longitude) de l'utilisateur. J'ai également mis en place une gestion des permissions pour garantir que l'application peut accéder à la position.

La fonction `getCurrentLocation` permet de récupérer les coordonnées (latitude et longitude) de l'utilisateur. Elle gère également les permissions et les erreurs liées au service de géolocalisation.

```
Future<LocationResult> getCurrentLocation() async {
  try {
    Location location = Location();

    bool serviceEnabled;
    LocationData locationData;

    serviceEnabled = await location.serviceEnabled();
    if (!serviceEnabled) {
      serviceEnabled = await location.requestService();
      if (!serviceEnabled) {
        return const LocationResult.error(LocationFailure.serviceDisabled);
      }
    }

    PermissionStatus permission = await location.hasPermission();
    if (permission == PermissionStatus.denied) {
      permission = await location.requestPermission();
    }
    if (permission == PermissionStatus.deniedForever) {
      return const LocationResult.error(
        LocationFailure.permissionDeniedForever,
      );
    }
    if (permission != PermissionStatus.granted) {
      return const LocationResult.error(LocationFailure.permissionDenied);
    }

    locationData = await location.getLocation();
    return LocationResult.success(locationData);
  } on PlatformException catch (e) {
    return const LocationResult.error(LocationFailure.unknown);
  }
}
```

Extrait du fichier `location_service.dart` pour la récupération de la géolocalisation de l'utilisateur

- Utilisation de `location.serviceEnabled()` pour vérifier si le GPS est activé.
- Si le service est désactivé, une demande d'activation est faite via `location.requestService()`.
- Si l'utilisateur refuse, une erreur sera retournée.
- Vérification des permissions avec `location.hasPermission()`.
- Si toutes les vérifications passent, les coordonnées sont récupérées via `location.getLocation()`.

Conversion des données en nom de ville :

Lorsque les coordonnées sont récupérées, la fonction `getCityFromCoordinates` les convertit en un nom de ville via le package `geocoding`. Cela permet d'afficher le nom dans l'interface au lieu de coordonnées.

```
Future<GeoPosition?> getCityFromCoordinates(
  double latitude,
  double longitude,
) async {
  try {
    List<Placemark> placemarks = await placemarkFromCoordinates(
      latitude,
      longitude,
    );
    final firstChoice = placemarks.first;
    return geocoding.GeoPosition(
      city: firstChoice.locality ?? 'Current Location',
      latitude: latitude,
      longitude: longitude,
    );
  } catch (e) {
    return null;
  }
}
```

Extrait du fichier `location_service.dart` pour convertir des coordonnées en nom de ville

Gestion des erreurs et affichage utilisateur :

Dans `home_view.dart`, la fonction `_showLocationError` gère les erreurs liées à la géolocalisation et affiche un `SnackBar` avec un message adapté au type d'erreur.

```
void _showLocationError(LocationFailure failure) {
  final messages = {
    LocationFailure.serviceDisabled:
      'La géolocalisation est désactivée sur votre appareil.',
    LocationFailure.permissionDenied: 'Permission de localisation refusée.',
    LocationFailure.permissionDeniedForever:
      'Permission refusée définitivement. Activez-la dans les paramètres de l\'application.',
    LocationFailure.unknown: 'Impossible d\'obtenir votre position.',
  };

  ScaffoldMessenger.of(context).showSnackBar(
    SnackBar(
      content: Text(messages[failure]!),
      backgroundColor: Colors.red[400],
      duration: const Duration(seconds: 4),
      action: failure == LocationFailure.permissionDeniedForever
        ? SnackBarAction(
            label: 'Paramètres',
            textColor: Colors.white,
            onPressed: () =>
              openAppSettings(), // package permission_handler
          ) // SnackBarAction
        : null,
    ), // SnackBar
  );
}
```

Extrait du fichier `home_view` pour la gestion d'affichage des erreurs de géolocalisation

- Messages d'erreur adaptées au type d'erreur
- Dans le cas où l'utilisateur a refusé la permission, un bouton "Paramètres" est affiché pour permettre à l'utilisateur de réactiver manuellement la permission.
- La Snackbar est visible pendant 4 secondes afin que l'utilisateur ai le temps de voir le message.

3 - Recherche et conversion des villes saisies par l'utilisateur :

Cette fonctionnalité permet à l'utilisateur de saisir manuellement une ville et d'afficher sa météo.

```
@override
Widget build(BuildContext context) {
  return Padding(
    padding: EdgeInsetsGeometry.only(left: 15),
    child: Row(
      children: [
        Expanded(
          child: TextField(
            controller: _controller,
            decoration: const InputDecoration(hintText: "Ajouter une ville"),
            onSubmitted: (_) =>
              _handleSubmit(),
            enabled: !_isLoading,
          ), // TextField
        ), // Expanded
        _isLoading
          ? const Padding(
              padding: EdgeInsets.all(12),
              child: SizedBox(
                width: 24,
                height: 24,
                child: CircularProgressIndicator(strokeWidth: 2),
              ), // SizedBox
            ) // Padding
          : IconButton(
              onPressed: _handleSubmit,
              icon: Icon(
                Icons.send,
                color: Theme.of(context).primaryColorDark,
              ), // Icon
            ), // IconButton
      ],
    ), // Row
  ); // Padding
}
```

Extrait du fichier `add_add_city_form.dart` qui contient le formulaire de recherche d'une ville

Conversion du nom de la ville en coordonnées :

La fonction `getCoordinatesFromCity` dans `location_service.dart` prend en entrée un nom de ville et retourne ses coordonnées géographiques. Si la ville est introuvable, elle retourne null.

```
Future<GeoPosition?> getCoordinatesFromCity(String city) async {
  try {
    final placemarks = await locationFromAddress(city);
    if (placemarks.isNotEmpty) {
      final firstChoice = placemarks.first;
      return geocoding.GeoPosition(
        city: city,
        latitude: firstChoice.latitude,
        longitude: firstChoice.longitude,
      );
    }
    return null;
  } on PlatformException catch (e) {
    return null;
  } catch (e) {
    return null;
  }
}
```

Extrait du fichier `location_service.dart` pour la récupération des coordonnées géographiques à partir d'un nom de ville

- `locationFromAddress` retourne une liste de `Placemark` correspondant au nom de la ville. Si la liste est vide, la ville est introuvable.

Validation des entrées et gestion des doublons :

Dans `home_view.dart`, la fonction `onhandleAddCity` gère l'ajout d'une ville saisie par l'utilisateur. Elle inclut :

- Une vérification du champ vide
- Une recherche des coordonnées via `getCoordinatesFromCity`
- Une vérification des doublons
- L'affichage de messages d'erreur via `SnackBar`

```
Future<void> onhandleAddCity(String city) async {
  if (city.trim().isEmpty) return;
  final GeoPosition? position = await LocationService()
    .getCoordinatesFromCity(city.trim());

  if (position == null) {
    // Ville introuvable
    if (mounted) {
      ScaffoldMessenger.of(context).showSnackBar(
        SnackBar(
          content: Text('Ville "$city" introuvable'),
          backgroundColor: Colors.red[400],
        ), // SnackBar
      );
    }
    return;
  }
  // Doublon
  if (cities
    .map((c) => c.toLowerCase())
    .contains(position.city.toLowerCase())) {
    if (mounted) {
      ScaffoldMessenger.of(context).showSnackBar(
        SnackBar(
          content: Text('${position.city} est déjà dans la liste'),
          backgroundColor: Colors.orange[400],
        ), // SnackBar
      );
    }
    return;
  }
  await onAddCity(position.city);
  _positionToCall = await LocationService().getCoordinatesFromCity(
    position.city,
  );
  callApi();
}
```

Extrait de `home_view.dart` pour la gestion de l'ajout d'une ville

DOSSIER PROFESSIONNEL (DP)

- Validation des données : `city.trim().isEmpty` évite les villes vides ou composées d'espaces et `getCoordinatesFromCity` vérifie que la ville existe.
- Gestion des doublons : on vérifie que la ville n'est pas déjà présente dans la liste des villes sauvegardées. Si c'est le cas, un message s'affiche pour avertir que la ville est déjà enregistrée.
- Après l'ajout de la ville, l'interface est mise à jour avec les informations de cette ville.



Affichage des prévisions après géolocalisation



Formulaire de saisie d'une ville

DOSSIER PROFESSIONNEL (DP)

13:58

Nantes

Ajouter une ville

Légère pluie
Min: 16.9°C
Max: 17.1°C

17°C

Jeudi

Légère pluie
Min: 11°C - Max: 17°C

Vendredi

Nuageux
Min: 8°C - Max: 21°C

Samedi

Nuageux
Min: 11°C - Max: 22°C

Dimanche

Légère pluie
Min: 13°C - Max: 21°C

Lundi

Légère pluie
Min: 12°C - Max: 15°C

Ville "azerty" introuvable

Message d'erreur pour ville introuvable

17:55

Rennes

Ajouter une ville

Nuageux
Min: 15.1°C
Max: 15.7°C

15°C

Jeudi

Nuageux
Min: 11°C - Max: 15°C

Vendredi

Peu nuageux
Min: 7°C - Max: 20°C

Samedi

Légère pluie
Min: 11°C - Max: 21°C

Dimanche

Légère pluie
Min: 12°C - Max: 17°C

Lundi

Couvert
Min: 7°C - Max: 12°C

"rennes" est déjà dans la liste

Message d'avertissement pour ville en doublon

4 - Sauvegarde de villes recherchées :

Le stockage des villes recherchées permet à l'utilisateur de sauvegarder ces villes et d'y accéder rapidement sans avoir à les rechercher à chaque ouverture de l'application. Cette fonctionnalité repose sur le package `shared_preferences` qui permet de stocker de petites quantités de données (comme une liste de noms de villes).

Implémentation du service de stockage :

Le service `DataService` (dans `data_service.dart`) gère la persistance des villes recherchées via trois fonctions principales :

- `getCities()` : Récupère la liste des villes sauvegardées.
- `addCity(String city)` : Ajoute une ville à la liste.
- `removeCity(String city)` : Supprime une ville de la liste.

```
import 'package:shared_preferences/shared_preferences.dart';

class DataService {
  final key = "cities";

  Future<List<String>> getCities() async {
    final preferences = await SharedPreferences.getInstance();
    final list = preferences.getStringList(key);
    return list ?? [];
  }

  Future<bool> addCity(String string) async {
    final prefs = await SharedPreferences.getInstance();
    var list = prefs.getStringList(key) ?? [];
    list.add(string);
    return prefs.setStringList(key, list);
  }

  Future<bool> removeCity(String string) async {
    final prefs = await SharedPreferences.getInstance();
    var list = prefs.getStringList(key) ?? [];
    list.remove(string);
    return prefs.setStringList(key, list);
  }
}
```

DataService qui gère la persistance des données grâce au package shared_preferences

- SharedPreferences permet de stocker des données simples
- La clé "cities" identifie la liste des villes dans le stockage

Intégration dans HomeView :

Dans home_view.dart, les méthodes de DataServices sont utilisées pour :

- Récupérer la liste des villes au démarrage de l'application
- Ajouter une ville après validation
- Supprimer une ville depuis le menu latéral

```
import 'package:shared_preferences/shared_preferences.dart';

class DataService {
  final key = "cities";

  Future<List<String>> getCities() async {
    final preferences = await SharedPreferences.getInstance();
    final list = preferences.getStringList(key);
    return list ?? [];
  }

  Future<bool> addCity(String string) async {
    final prefs = await SharedPreferences.getInstance();
    var list = prefs.getStringList(key) ?? [];
    list.add(string);
    return prefs.setStringList(key, list);
  }

  Future<bool> removeCity(String string) async {
    final prefs = await SharedPreferences.getInstance();
    var list = prefs.getStringList(key) ?? [];
    list.remove(string);
    return prefs.setStringList(key, list);
  }
}
```

Extrait de `home_view` pour la gestion des villes sauvegardées

- `updateCities` met à jour la liste `cities` et rafraîchit l'interface avec `setState`
- `onAddCity` et `removeCity` appellent `DataService` puis mettent à jour la liste via `updateCities`

5 - Gestion de l'état global dans `HomeView` :

Le widget `HomeView` est un `StatefulWidget` qui gère plusieurs états :

- `_userPosition` : Position géolocalisée de l'utilisateur qui est récupérée au démarrage.
- `_positionToCall` : Position (ville) actuellement sélectionnée pour afficher la météo
- `cities` : Liste des villes favorites sauvegardées
- `apiResponse` : Données météo retournées par l'API `OpenWeather`
- `_isLoading` : Indiquant si la géolocalisation ou l'appel API sont en cours.

```
@override
void initState() {
  getUserLocation();
  updateCities();
  super.initState();
}
```

Extrait de `home_view.dart`

Au démarrage de l'application, la fonction `initState` appelle `getUserLocation` pour récupérer la position de l'utilisateur et `updateCities` pour charger la liste des villes sauvegardées.

Récupération de la position utilisateur et appel à l'API :

La fonction `getUserLocation` gère la géolocalisation et l'appel à l'API pour afficher la météo de la position actuelle. Elle utilise `_isLoading` pour indiquer qu'une opération est en cours.

```
Future<void> getUserLocation() async {
  setState(() => _isLoading = true);

  final result = await LocationService().getCurrentLocation();
  if (!result.isSuccess) {
    if (mounted) {
      _showLocationError(result.failure!);
      setState(() => _isLoading = false);
    }
    return;
  }

  final latitude = result.data!.latitude!;
  final longitude = result.data!.longitude!;

  final loc = await LocationService().getCityFromCoordinates(latitude, longitude);
  if (loc != null) {
    setState(() {
      _userPosition = loc;
      _positionToCall = loc;
    });
    callApi();
  }

  if (mounted) setState(() => _isLoading = false);
}
```

Extrait de `home_view.dart` pour récupérer la géolocalisation et les données météo

- Si `_isLoading` est égale à `true` un indicateur de chargement des données est affiché
- `mounted` vérifie que le widget est toujours monté avant d'appeler `setState`

Appel à l'API et mise à jour de l'interface :

La fonction `callApi` récupère les données météo pour la ville sélectionnée (`_positionToCall`) et met à jour l'interface avec le résultat.

```
Future<void> callApi() async {
  if (_positionToCall == null) return;
  final response = await ApiService().callApi(_positionToCall!);
  if (mounted) {
    setState(() {
      apiResponse = response;
    });
  }
}
```

Extrait de `home_view.dart` pour récupérer les données de l'API en fonction de la ville

- Si `positionToCall` est `null`, aucune ville n'est sélectionnée donc l'appel API n'est pas déclenché.
- `setState` met à jour `apiResponse` et déclenche un nouveau rendu de l'interface.

Sélection d'une ville et mise à jour :

Lorsque l'utilisateur sélectionne une ville via le Drawer, la fonction `onTap` met à jour `_positionToCall` et appelle `callApi()` pour afficher la météo de la ville sélectionnée.

```
Future<void> onTap(String citySelected) async {
  Navigator.of(context).pop();
  removeKeyboard();
  if (citySelected == _userPosition?.city) {
    _positionToCall = _userPosition;
    callApi();
  } else {
    _positionToCall = await LocationService().getCoordinatesFromCity(citySelected);
    callApi();
  }
}
```

Extrait de `home_view.dart` pour gérer le changement de ville

- Lors de sélection d'une ville, le Drawer (menu latéral) se ferme grâce à `Navigator.of(context).pop()`
- `_positionToCall` est mis à jour, si la ville sélectionnée correspond à la ville de l'utilisateur alors on utilise cette position, sinon on récupère la position via la fonction `getCoordinatesFromCity`

Affichage conditionnel dans l'interface :

Le widget `build` de `HomeView` utilise des conditions pour afficher soit :

- Un indicateur de chargement

- Un message indiquant qu'il n'y a aucune donnée
- Les données météo

```
@override
Widget build(BuildContext context) {
  return Scaffold(
    appBar: AppBar(
      title: Text(
        _positionToCall?.city.capitalize ??
        "Ma météo",
      ), // Text
    ), // AppBar
    drawer: MyDrawer(
      myPosition: _userPosition,
      cities: cities,
      onTap: onTap,
      onDelete: removeCity,
      selectedCity: _positionToCall?.city,
    ), // MyDrawer
    body: SafeArea(
      child: Column(
        children: [
          AddCityForm(onAddCity: onhandleAddCity),
          Expanded(
            child: _isLoading
              ? const Center(child: CircularProgressIndicator())
              : apiResponse == null
                ? NoData()
                : ForecastData(response: apiResponse!),
          ), // Expanded
        ],
      ), // Column
    ), // SafeArea
  ); // Scaffold
}
```

Widget de `home_view.dart`

6 - Le menu latéral et la mise en avant de la ville sélectionnée :

Le widget `MyDrawer` est un menu latéral qui permet à l'utilisateur d'accéder à ses villes sauvegardées et à sa position actuelle. L'un de ses points forts est la mise en avant visuelle de la ville actuellement sélectionnée, ce qui améliore la clarté et l'expérience utilisateur.

Organisation du menu :

Le Drawer est structuré en trois parties :

- En haut une icône de localisation et le titre "Mes Villes"
- La position actuelle de l'utilisateur (si disponible)
- La liste des villes sauvegardées avec la possibilité de les sélectionner ou de les supprimer

```
@override
Widget build(BuildContext context) {
  final itemCount = (myPosition == null)
    ? cities.length + 1
    : cities.length + 2;
  return Drawer(
    child: ListView.separated(
      itemBuilder: ((context, index) {
        if (index == 0) return header(context);
        if (index == 1 && myPosition != null) {
          return currentPositionTitle(myPosition!.city, context);
        }
        if (myPosition == null) {
          return citySavedTitle(cities[index - 1], true, context);
        }
        return citySavedTitle(cities[index - 2], true, context);
      }),
      separatorBuilder: (context, index) {
        if (index == 0) {
          return const SizedBox.shrink();
        }
        return const Divider();
      },
      itemCount: itemCount,
    ), // ListView.separated
  ); // Drawer
}
```

Widget de *my_drawer.dart*

- `ListView.separated` : Permet d'afficher une liste avec des séparateurs (`Divider`) entre les éléments.
- Les index gèrent le positionnement des éléments

Mise en avant de la ville sélectionnée :

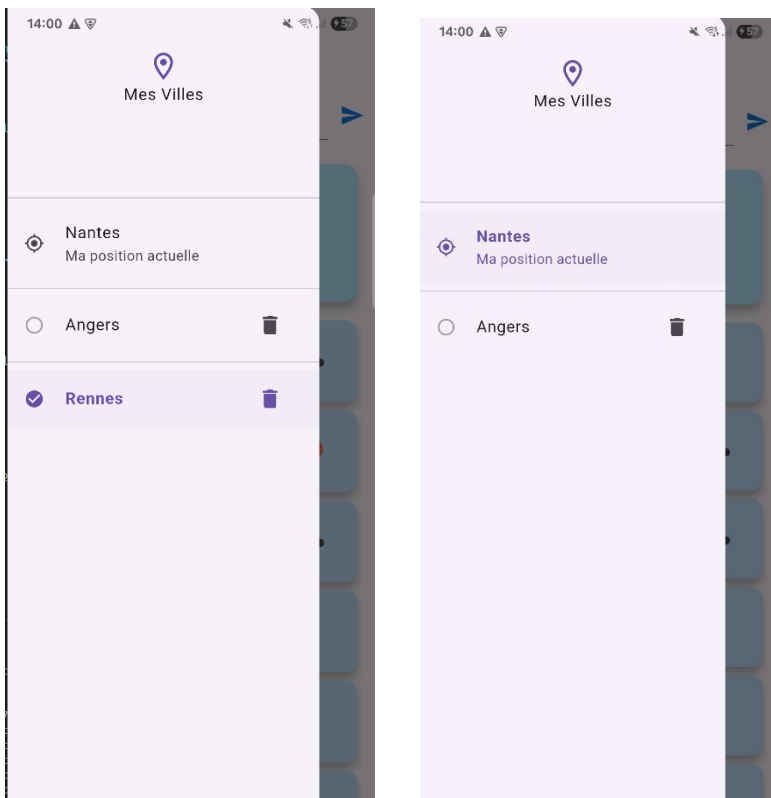
Le paramètre `selectedCity` est passé au widget `MyDrawer` depuis `HomeView`. On va alors comparer cette valeur avec la valeur de chaque ville de la liste. Si ces valeurs sont identiques alors une icône est rajoutée et le nom de la ville est en gras et d'une autre couleur.

```
ListTile currentPositionTitle(String city, BuildContext context) {
  final isSelected = selectedCity == city;
  return ListTile(
    leading: const Icon(Icons.my_location, size: 20),
    title: Text(
      city,
      style: isSelected
        ? TextStyle(
            fontWeight: FontWeight.bold,
            color: Theme.of(context).primaryColor,
          ) // TextStyle
        : null,
    ), // Text
    subtitle: const Text("Ma position actuelle"),
    onTap: () => onTap(city),
    selected: isSelected,
    selectedTileColor: Theme.of(context).primaryColor.withAlpha(10),
  ); // ListTile
}
```

Extrait de *my_drawer.dart* pour afficher la position actuelle de l'utilisateur dans le menu latéral

```
ListTile citySavedTile(String city, bool canDelete, BuildContext context) {  
  final isSelected = selectedCity == city;  
  return ListTile(  
    title: Text(  
      city.capitalize,  
      style: isSelected  
        ? TextStyle(  
          fontWeight: FontWeight.bold,  
          color: Theme.of(context).primaryColor,  
        ) // TextStyle  
        : null,  
    ), // Text  
    onTap: (() => onTap(city)),  
    selected: isSelected,  
    selectedTileColor: Theme.of(context).primaryColor.withAlpha(10),  
    leading: isSelected  
      ? Icon(  
        Icons.check_circle,  
        color: Theme.of(context).primaryColor,  
        size: 20,  
      ) // Icon  
      : const Icon(Icons.circle_outlined, size: 20, color: Colors.grey),  
    trailing: (canDelete)  
      ? IconButton(  
        onPressed: (() => onDelete(city)),  
        icon: const Icon(Icons.delete),  
      ) // IconButton  
      : null,  
  ); // ListTile  
}
```

Extrait de `my_drawer.dart` pour afficher une ville sauvegardée dans le menu latéral



Affichage du menu latéral avec différentes villes sélectionnées

Tests et validation :

J'ai réalisé des tests manuels pour valider le bon fonctionnement de l'application dont voici les scénarios :

- Test de géolocalisation automatique et affichage de la météo locale
- Test d'ajout d'une ville valide
- Test de tentative d'ajout d'une ville introuvable
- Test de tentative d'ajout d'un doublon
- Test de sélection d'une ville depuis le menu latéral

Compétences mobilisées :

Conception et développement d'interfaces utilisateur :

- Choix des widgets : Utilisation de widgets Flutter adaptés : AppBar, Drawer, ListTile, Card, TextField, IconButton
- Responsive Design : Adaptation de l'UI aux différentes tailles d'écran avec MediaQuery, Expanded, Flexible.
- Expérience utilisateur : Mise en avant visuelle de la ville sélectionnée
- Navigation intuitive : Utilisation d'un Drawer pour accéder aux villes sauvegardées et basculer entre elles.

Intégration de données dynamiques :

- Appel à une API externe : Consommation de l'API OpenWeather pour récupérer les prévisions météorologiques
- Parsing des données JSON : Conversion des réponses JSON de l'API en objets Dart
- Affichage dynamique : Mise à jour de l'interface en fonction des données reçues
- Gestion des états : Utilisation de setState pour rafraîchir l'interface après un appel API ou une sélection de ville.

Gestion des interactions utilisateur :

- Gestion des entrées : Validation du formulaire de saisie d'une ville
- Sélection d'éléments : Sélection d'une ville via le Drawer ou la recherche avec mise à jour de l'interface
- Suppression d'éléments : Suppression d'une ville sauvegardée via le Drawer
- Feedback visuel : Affichage de SnackBar pour confirmer ou signaler une erreur

Gestion des erreurs :

- Gestion des permissions : Vérification et demande des permissions de géolocalisation avec redirection vers les paramètres si refus définitif
- Gestion des erreurs : Affichage de messages d'erreur via un SnackBar pour informer l'utilisateur

Stockage local et persistance des données

- Stockage des villes recherchées : Utilisation de SharedPreferences pour sauvegarder la liste des villes recherchées
- Persistance des données : Les villes sont conservées entre les sessions de l'application

Difficultés rencontrées :

- Gestion des permissions de géolocalisation :
Solution : Utilisation de LocationResult et LocationFailure pour typer les erreurs et afficher des messages adaptés
- Parsing des données JSON complexes :
Solution : Création de classes dédiées avec des constructeurs fromJson

2. Précisez les moyens utilisés :

Langage utilisé :

- Dart (3.10.0)

Framework utilisé

- Flutter (3.38.1) pour une application cross-platform (Android/iOS)

Packages utilisés

- Location pour récupérer les **coordonnées GPS de l'utilisateur**
- Geocoding pour convertir les coordonnées en ville et inversement
- http pour effectuer les requêtes http vers l'API
- shared_preferences pour stocker des données locales simples et de manière persistante
- permission_handler pour gérer les permissions

API utilisée :

- OpenWeather avec clé de sécurité

Éditeur de code :

- Visual Studio Code

Versionning

- Git Flow pour suivre et organiser les modifications
- GitHub pour héberger le code source

DOSSIER PROFESSIONNEL (DP)

Tests

- Tests manuels sur émulateurs et sur un appareil réel (Galaxy S21)
- Mode responsive avec le Toogle Device Toolbar de Chrome

3. Avec qui avez-vous travaillé ?

J'ai travaillé en autonomie sur ce projet en appliquant les compétences acquises durant la formation.

Ressources :

- Support de cours
- Documentation officielle Flutter
- Dépôt de packages officiel pour Flutter et Dart

4. Contexte

Nom de l'entreprise, organisme ou association ► **Global Digital University**

Chantier, atelier, service ► **Utiliser une API avec Flutter**

Période d'exercice ► **Du 08/12/2025 au 18/12/2025**

5. Informations complémentaires (facultatif)

DOSSIER PROFESSIONNEL (DP)

Activité-type 1 Développer une application sécurisée

CP 3 ► Développer des composants métier

1. Décrivez les tâches ou opérations que vous avez effectuées, et dans quelles conditions :

Dans le cadre de ma formation, j'ai conçu une API sécurisée de partage de projets destinée aux développeurs nommée DevConnect.

Objectif :

Créer une API sécurisée permettant de partager des projets et d'interagir avec les projets des autres utilisateurs de la plateforme.

Public Cible :

Développeurs qui souhaitent partager leur projet et découvrir ceux des autres

Fonctionnalités principales :

- Inscription et connexion d'un utilisateur
- Ajout, modification et suppression de projets
- Récupération des informations des projets
- Système de commentaire
- Système de like

Technologies :

- Node.js : environnement d'exécution JavaScript côté serveur
- Express : framework pour la création des routes et la gestion des requêtes http
- MongoDB : Base de données NoSQL
- Mongoose : ODM pour interagir avec MongoDB et définir les schémas de données

Architecture du projet

Pour structurer mon projet, j'ai choisi une architecture qui sépare les responsabilités entre les modèles de données, les controllers, les routes, les middlewares et la configuration.

Cela facilite la maintenance car chaque composant est isolé et peut être modifié indépendamment. Cela permet également d'assurer la scalabilité car l'application peut évoluer sans remettre en cause sa structure globale.

DOSSIER PROFESSIONNEL (DP)

```
dev-connect/
├─ node_modules/
├─ src/
│  ├─ config/
│  │  └─ database.js
│  ├─ controllers/
│  │  ├─ projects.controller.js
│  │  └─ users.controller.js
│  ├─ middlewares/
│  │  ├─ auth.js
│  │  └─ multer.js
│  ├─ models/
│  │  ├─ comment.model.js
│  │  ├─ project.model.js
│  │  └─ user.model.js
│  └─ routes/
│     ├─ projects.routes.js
│     └─ users.model.js
└─ app.js
├─ uploads/
├─ tests/
├─ .env
├─ .gitignore
├─ package.json
├─ swagger.js
└─ vercel.json
```

Fichier	Description
node_modules/	Dépendances npm
config/database.js	Configuration de la connexion à MongoDB
controllers/projects.controller.js	Logique métier pour les projets
controllers/users.controller.js	Logique métier pour les utilisateurs
middlewares/auth.js	Middleware d'authentification JWT
middlewares/multer.js	Middleware pour l'upload des images
models/comment.model.js	Schéma MongoDB pour les commentaires
models/project.model.js	Schéma MongoDB pour les projets
models/user.model.js	Schéma MongoDB pour les utilisateurs
routes/projects.route.js	Routes pour les projets
routes/users.route.js	Routes pour les utilisateurs
app.js	Point d'entrée de l'application
uploads/	Dossier de stockage des images uploadées
tests/	Tests unitaires et d'intégration
.env	Variables d'environnement
.gitignore	Fichiers exclus du versionnage Git
package.json	Métadonnées du projet, scripts et dépendances npm
swagger.js	Configuration de Swagger pour la documentation
vercel.json	Configuration pour le déploiement

1 – Modélisation des données

La modélisation des données permet de définir la structure des informations stockées dans la base de données MongoDB. Cette modélisation repose sur trois entités principales :

- User (Utilisateur) représente un compte utilisateur avec ses informations personnelles et ses identifiants
- Project (Projet) représente un projet publié par un utilisateur, avec ses informations (titre,

description, compétences, image)

- Comment (Commentaire) représente un commentaire laissé par un utilisateur sur un projet

L'objectif est de concevoir des schémas garantissant l'intégrité des données via des validations et des contraintes. Ces schémas facilitent également les requêtes grâce à des relations claires entre les modèles.

Avant d'implémenter les schémas MongoDB, une analyse des besoins a été réalisée pour identifier les entités, leurs attributs, et leurs relations.

Pour visualiser ces relations, un diagramme de classe a été conçu avec Draw.io. Il permet de clarifier la structure des données et les liens entre les entités.

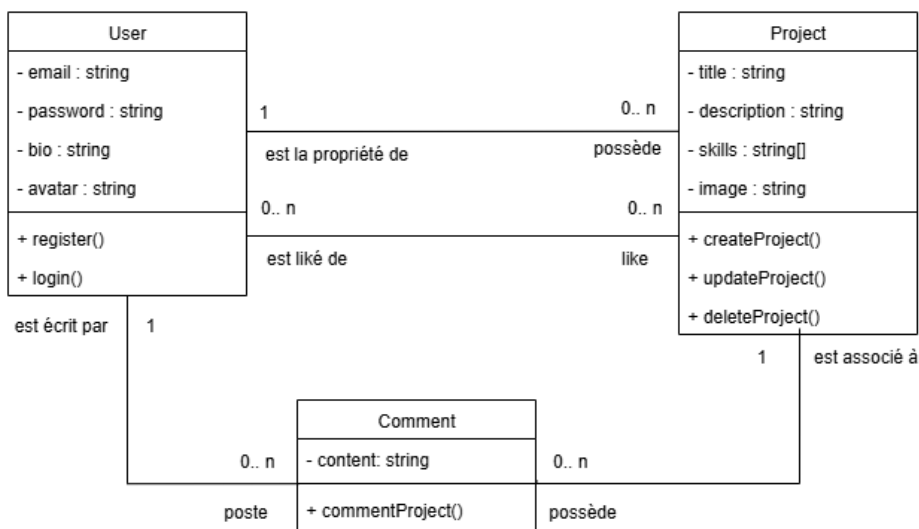


Diagramme de classe

Implémentation des schémas MongoDB avec Mongoose :

Les schémas ont été implémentés en utilisant **Mongoose**, un ODM (Object Data Modeling) pour MongoDB. Mongoose permet de :

- Définir des schémas stricts pour les documents MongoDB
- Ajouter des validations
- Gérer les références entre collections via ref

Le modèle User représente un utilisateur de l'application. Il inclut des champs pour l'authentification (email, password) et des informations optionnelles (bio, avatar).

```
const mongoose = require('mongoose');

const userSchema = new mongoose.Schema({
  email: { type: String, required: true, unique: true },
  password: { type: String, required: true },
  bio: { type: String, default: '' },
  avatar: { type: String }
}, { timestamps: true });

module.exports = mongoose.model('User', userSchema);
```

Schéma Mongoose du modèle User

Le modèle Project représente un projet publié par un utilisateur. Il inclut des champs pour les informations du projet (title, description, skills, image) et des références à l'auteur et aux interactions (likes, comments).

```
const mongoose = require('mongoose');

const projectSchema = new mongoose.Schema({
  title: { type: String, required: true },
  description: { type: String },
  skills: { type: [String], default: [] },
  image: { type: String },
  author: { type: mongoose.Schema.Types.ObjectId, ref: 'User', required: true },
  likes: [{ type: mongoose.Schema.Types.ObjectId, ref: 'User' }],
  comments: [{ type: mongoose.Schema.Types.ObjectId, ref: 'Comment' }]
}, { timestamps: true });

module.exports = mongoose.model('Project', projectSchema);
```

Schéma Mongoose du modèle Project

Le modèle Comment représente un commentaire laissé par un utilisateur sur un projet. Il inclut le contenu du commentaire et des références à l'auteur et au projet.

```
const mongoose = require('mongoose');

const commentSchema = new mongoose.Schema({
  content: { type: String, required: true },
  author: { type: mongoose.Schema.Types.ObjectId, ref: 'User', required: true },
  project: [{ type: mongoose.Schema.Types.ObjectId, ref: 'Project', required: true }]
}, { timestamps: true });

module.exports = mongoose.model('Comment', commentSchema);
```

Schéma Mongoose du modèle Comment

2 - La logique métier

Validations des données à partir du schéma :

Mongoose permet d'ajouter des validations directement dans les schémas : `required` pour les champs obligatoires, `unique` pour une valeur unique (par exemple l'email pour User) et `default` pour attribuer une valeur par défaut.

Validations des données dans les controllers :

Les controllers incluent des validations supplémentaires pour répondre aux besoins métier et garantir l'intégrité des données.

Dans la fonction register du fichier users.controller.js, j'ai utilisé le package password-validator pour créer un schéma de validation du mot de passe lors de la création du compte utilisateur.

```
const schema = new passwordValidator();
schema.is().min(8).has().uppercase().has().lowercase().has().digits().has().not().spaces();
if(!schema.validate(password)){
  return res.status(400).json({ message: 'Password must be at least 8 characters long, contain uppercase and lowercase letters, digits, and no spaces' });
}
```

Extrait de users.controller.js - schéma de validation du mot de passe

Dans la fonction createProject du fichier projects.controller.js, un contrôle est fait pour vérifier que corps de la requête contient le titre du projet qui est obligatoire. Le cas échéant une erreur 400 est retournée avec un message d'erreur.

```
exports.createProject = async (req, res, next) => {
  try{
    const {title, description, skills} = req.body;
    const image = req.file ? req.file.filename : null;
    if(!title){
      return res.status(400).json({ message: 'Title is required' });
    }
  }
}
```

Extrait de projects.controller.js – contrôle de la présence d'un titre de projet dans le corps de la requête POST

Dans la fonction getProject du fichier projects.controller.js, l'id qui est passée dans la query est contrôlé car il doit être un objectId MongoDB valide.

```
exports.getProject = async (req, res, next) => {
  const { id } = req.params;
  if(!mongoose.Types.ObjectId.isValid(id)){
    return res.status(400).json({ message: 'ID not valid' });
  }
}
```

Extrait de projects.controller.js – vérification que l'id passée dans la query est valide

Règles métier spécifiques :

Dans le cadre du système de like sur les projets, un utilisateur ne peut liker qu'une seule fois un projet donc si le projet est déjà like cela fera l'action inverse et retirera le like. Une seule fonction gère le fait d'ajouter ou retirer un like car cela évite de dupliquer du code étant donné que la logique est la même dans les deux cas.

```
const alreadyLiked = project.likes.includes(userId);
if(alreadyLiked){
  project.likes = project.likes.filter(uId => uId.toString() !== userId);
}else{
  project.likes.push(userId);
}
```

Extrait de projects.controller.js – gestion du like d'un projet pour un utilisateur

Autorisations :

Un utilisateur ne peut modifier ou supprimer que ses propres projets. Cette règle est vérifiée dans les fonctions updateProject et deleteProject du fichier projects.controller.js.

```
if(project.author.toString() !== req.user.userId){
  return res.status(403).json({ message: 'Not authorized' });
}
```

Extrait de *projects.controller.js* – contrôle que l'utilisateur connecté est le propriétaire du projet

Gestion des commentaires :

Un commentaire ne doit pas avoir un contenu vide et il doit être associé à un projet existant.

```
exports.commentProject = async (req, res, next) => {
  try{
    const {id} = req.params;
    const userId = req.user.userId;
    const { content } = req.body;

    const project = await Project.findById(id);
    if(!project){
      return res.status(404).json({ message: 'Project not found' });
    }
    if(!content){
      return res.status(400).json({ message: 'Content is required' });
    }
  }
}
```

Extrait de *projects.controller.js* – contrôle lors de l'ajout d'un commentaire sur un projet

3 - L'intégration avec la base de données

L'intégration avec MongoDB est réalisée via Mongoose, qui permet d'effectuer des opérations CRUD (Create, Read, Update, Delete) de manière simple et sécurisée. Les requêtes sont optimisées pour répondre aux besoins de l'application.

Opérations CRUD sur les projets :

Dans la fonction *createProject*, l'objet *Project* est créé avec le champ *author* automatiquement rempli avec *id* de l'utilisateur connecté. La requête MongoDB est exécutée via la fonction *save* qui insère la donnée en base.

```
const project = new Project({
  title,
  description,
  skills,
  image,
  author: req.user.userId,
  likes: [],
  comments: [],
});
res.status(201).json(await project.save());
```

Extrait de *projects.controller.js* –requête MongoDB de création d'un projet

Dans la fonction `getProjects` de récupération des projets, les objets `Project` retournés contiennent uniquement le titre et la description via le `select`, et les informations de l'auteur (email, bio et avatar) via le `populate` qui est utilisé lorsqu'il s'agit de modèle référencé. Le `sort` sert à trier les résultats.

```
const projects = await Project.find( )
  .select('title description')
  .populate('author', 'email bio avatar')
  .sort({ createdAt: -1 });
```

Extrait de `projects.controller.js` – requête MongoDB de récupération des projets

Dans la fonction `getProject`, un projet spécifique est récupéré via le `findById`. Grâce au `populate`, les `ObjectId` sont remplacés par les documents des modèles référencés.

```
const project = await Project.findById(id)
  .populate('author', 'email bio avatar')
  .populate({
    path: 'comments',
    populate: {
      path: 'author',
      select: 'email avatar'
    }
  });
```

Extrait de `projects.controller.js` – requête MongoDB de récupération d'un projet spécifique

Dans la fonction `updateProject`, le `findById` permet de récupérer le projet à partir de l'id passé dans la requête. Les informations du `body` sont récupérées et seuls les champs fournis dans la requête sont mis à jour.

```
const { title, description, skills } = req.body;
const image = req.file ? req.file.filename : null;
if(title) project.title = title;
if(description) project.description = description;
if(skills) project.skills = skills;
if(image) project.image = image;

res.status(201).json(await project.save());
```

Extrait de `projects.controller.js` – requête MongoDB de mise à jour d'un projet

Dans la fonction `deleteProject`, la fonction `deleteOne` permet de supprimer le projet.

```
const {id} = req.params;
const project = await Project.findById(id);
if(!project){
  return res.status(404).json({ message: 'Project not found' });
}
if(project.author.toString() !== req.user.userId){
  return res.status(403).json({ message: 'Not authorized' });
}
await project.deleteOne()
res.status(200).json({message:'Project deleted'});
```

Extrait de `projects.controller.js` – requête MongoDB de suppression d'un projet

Pour l'ajout d'un commentaire sur un projet, la fonction save sur l'objet Comment permet d'insérer un nouveau commentaire en base et la fonction save sur l'objet Project permet de le mettre à jour après lui avoir ajouté le commentaire.

```
const comment = new Comment({
  content,
  author: userId,
  project: id
});
const savedComment = await comment.save();
project.comments.push(savedComment._id);
await project.save();
```

Extrait de projects.controller.js – ajout d'un commentaire associé à un projet

4 - La sécurité

La sécurité est un aspect critique pour une API qui gère des données sensibles (mots de passe, informations utilisateur). J'ai mis en place plusieurs mesures pour protéger l'application et garantir la confidentialité des données.

Hachage des mots de passe :

J'ai utilisé le package bcrypt pour hacher le mot de passe dans la fonction Register qui gère l'inscription d'un utilisateur. C'est la version hachée qui sera sauvegardée en base de données pour garantir la sécurité du mot de passe.

```
const hashPassword = await bcrypt.hash(password, 10);
const newUser = new User({
  email,
  password: hashPassword,
  bio,
  avatar
});
```

Extrait de users.controller.js – hachage du mot de passe

Dans la fonction Login qui gère la connexion de l'utilisateur, la fonction bcrypt.compare permet de comparer le mot de passe saisi par l'utilisateur avec le mot de passe haché stocké.

```
const isPasswordCorrect = await bcrypt.compare(password, existingUser.password);
if(!isPasswordCorrect){
  return res.status(400).json({ message: 'Credentials invalid' });
}
```

Extrait de users.controller.js – comparaison entre le mot de passe fourni et le mot de passe haché stocké

Authentification avec JWT :

J'ai utilisé le package jsonwebtoken pour générer un token contenant l'id de l'utilisateur et la date d'expiration du token. Ce token est envoyé en réponse de la requête de connexion de l'utilisateur, le client

redevra ensuite envoyer ce token dans les headers des requêtes pour avoir l'autorisation d'accéder à l'API. La clé secrète (JWT_SECRET) est stockée dans les variables d'environnement (.env) pour la sécurisée.

```
const token = jwt.sign({ userId: existingUser._id }, process.env.JWT_SECRET, { expiresIn: '7d' });
res.status(200).json({ user: existingUser, token });
res.end();
```

Extrait de *users.controller.js* – génération du token

Middleware d'authentification :

J'ai créé un middleware d'authentification qui est appliqué aux routes protégées. Il permet de vérifier la présence d'un token dans les headers et que celui-ci est valide.

```
const jwt = require('jsonwebtoken');

const auth = (req, res, next) => {
  const token = req.header('Authorization')?.split(' ')[1];
  if(!token){
    return res.status(401).json({ message: 'No token, authorization denied' });
  }
  try{
    const decodedUser = jwt.verify(token, process.env.JWT_SECRET);
    req.user = decodedUser;
    next();
  }catch(error){
    return res.status(401).json({ message: 'Token is not valid' });
  }
}

module.exports = auth;
```

Middleware *auth.js* pour vérifier le token JWT

```
router.post('/', auth, upload.single('image'), createProject);
```

Route de création d'un projet avec le middleware d'authentification appliqué

Protection contre les attaques :

J'ai utilisé le package *helmet* car il ajoute des en-têtes HTTP sécurisés pour protéger contre les attaques comme Cross-Site Scripting (XSS) et Clickjacking.

```
const helmet = require('helmet');
app.use(helmet());
```

Extrait de *app.js*

Limitation des requêtes :

J'ai utilisé le package *express-rate-limit* pour la protection contre les attaques par brute-force en limitant le nombre de requêtes par IP. Ces attaques consistent à envoyer des milliers de requêtes avec différentes

combinaisons pour trouver des mots de passe, clé API, token...

```
const rateLimit = require('express-rate-limit');
app.use(rateLimit({
  windowMs: 15 * 60 * 1000, // 15 minutes
  max: 100 // limit each IP to 100 requests per windowMs
}));
```

Extrait de app.js

5- Construction des routes API

Les routes définissent les endpoints de l'API qui permettent aux clients d'interagir avec les fonctionnalités de l'application. Elles sont organisées dans le dossier `routes/` et utilisent les controllers pour traiter les requêtes. Les routes sont séparées par ressource pour une meilleure maintenabilité.

Définition des routes :

Dans app.js, les routes sont préfixées

```
const projetRoutes = require('./routes/projects.routes');
const userRoutes = require('./routes/users.routes');

app.use('/api/projects', projetRoutes);
app.use('/api/users', userRoutes);
```

Extrait de app.js

Dans le fichier projects.routes, on retrouve les endpoints lié à l'entité Project :

Les routes CRUD :

- GET / : Récupérer tous les projets
- GET /:id : Récupérer un projet spécifique
- POST / : Créer un projet
- PUT /:id : Mettre à jour un projet
- DELETE /:id : Supprimer un projet

Mais aussi des routes spécifiques :

- PATCH /:id/like : Ajouter/retirer un like à un projet
- POST /:id/comments : Ajouter un commentaire à un projet

Le middleware auth est appliqué sur toutes les routes pour protéger leurs accès.

Le middleware multer est utilisé sur les routes de création et modification d'un projet pour gérer l'upload de l'image du projet.

```
const express = require('express');
const { getProjects, getProject, createProject, updateProject, deleteProject, likeProject, commentProject } = require('../controllers/projects.controller');
const router = express.Router();
const auth = require('../middlewares/auth');
const upload = require('../middlewares/multer');

router.get('/', getProjects);
router.get('/:id', getProject);
router.post('/', auth, upload.single('image'), createProject);
router.put('/:id', auth, upload.single('image'), modifyProject);
router.delete('/:id', auth, deleteProject);
router.post('/:id/like', auth, likeProject);
router.post('/:id/comment', auth, commentProject);
```

Liste des routes dans projects.routes.js

Documentation Swagger :

Pour faciliter l'utilisation et le test de l'AP, une documentation interactive a été générée avec Swagger UI. Cette documentation permet de :

- Voir les endpoints disponibles
- Tester directement les requêtes depuis l'interface web
- Consulter les schémas de données
- Voir les exigences de sécurité

La configuration de Swagger est centralisée dans le fichier `swagger.js` qui définit :

- Les métadonnées de l'API (titre, version, description)
- Les schémas de données
- Les schémas de sécurité (authentification par JWT)
- Les sources des annotations (fichiers de routes)

```
const swaggerUi = require('swagger-ui-express');
const swaggerJsDoc = require('swagger-jsdoc');

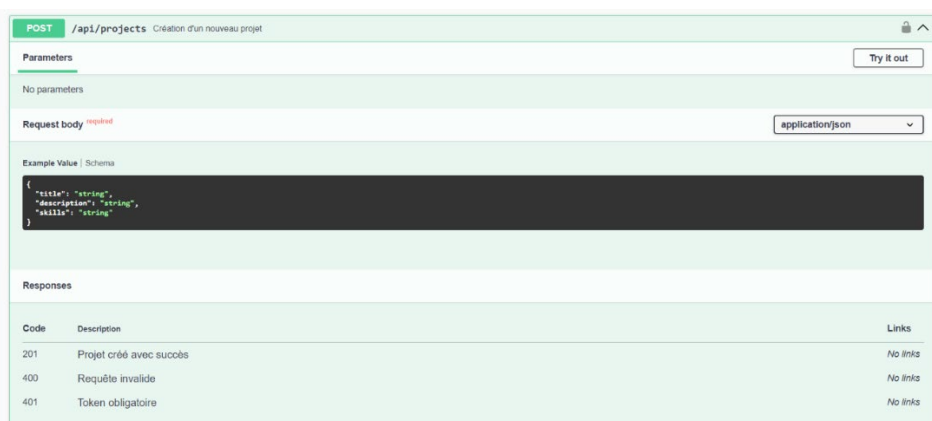
const options = {
  definition: {
    openapi: '3.0.0',
    info: {
      title: 'API de gestion de projets',
      version: '1.0.0',
      description: 'Documentation de l\'API permettant de gérer les projets, les commentaires et les utilisateurs.'
    },
  },
  components: {
    securitySchemes: {
      bearerAuth: {
        type: 'http',
        scheme: 'bearer',
        bearerFormat: 'JWT'
      }
    },
  },
  schemas: [
    {
      $id: 'Project',
      type: 'object',
      required: ['title', 'author'],
      properties: {
        _id: { type: 'string', description: 'ID du projet' },
        title: { type: 'string', description: 'Titre du projet' },
        description: { type: 'string', description: 'Description du projet' },
        skills: { type: 'array', items: { type: 'string', description: 'Liste des compétences ou technologies utilisées' } },
        image: { type: 'string', description: 'URL de l\'image du projet' },
        author: { type: 'string', description: 'ID de l\'utilisateur auteur du projet' },
        likes: { type: 'array', items: { type: 'string', description: 'IDs des utilisateurs qui ont aimé ce projet' } },
        comments: { type: 'array', items: { type: 'string', description: 'IDs des commentaires liés à ce projet' } },
        createdAt: { type: 'string', format: 'date-time', description: 'Date de création du projet' },
        updatedAt: { type: 'string', format: 'date-time', description: 'Date de la dernière mise à jour du projet' }
      }
    }
  ],
};
```

Extrait du fichier `swagger.js` - configuration de la documentation SwaggerUI

Les annotations sur chaque route permettent de générer automatiquement la documentation. Sur la route d'ajout de projet, la documentation inclut un schéma de sécurité pour l'authentification par JWT.

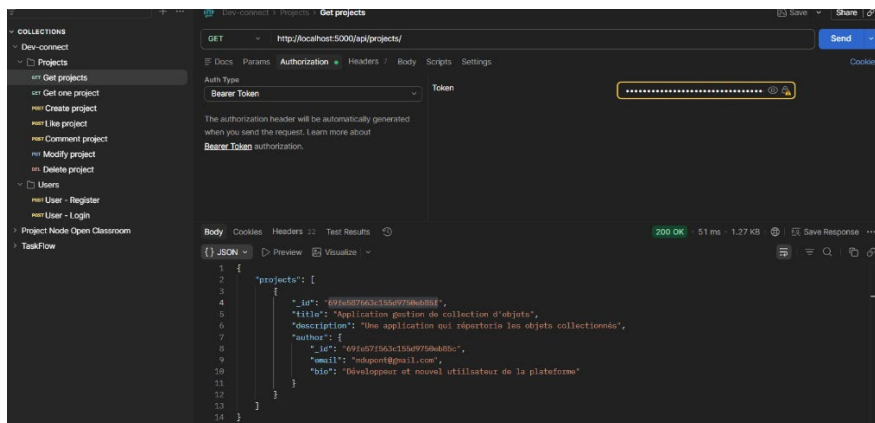
```
/**
 * @swagger
 * /api/projects:
 *   post:
 *     summary: Création d'un nouveau projet
 *     tags: [Projects]
 *     security:
 *       - bearerAuth: []
 *     requestBody:
 *       required: true
 *       content:
 *         application/json:
 *           schema:
 *             type: object
 *             required:
 *               - title
 *             properties:
 *               title:
 *                 type: string
 *               description:
 *                 type: string
 *               skills:
 *                 type: string
 *     responses:
 *       201:
 *         description: Projet créé avec succès
 *       400:
 *         description: Requête invalide
 *       401:
 *         description: Token obligatoire
 */
router.post('/', auth, upload.single('image'), createProject);
```

Extrait du fichier `projects.routes.js` – endpoint d'ajout d'un projet avec les annotations Swagger



Interface de la documentation SwaggerUI sur l'endpoint d'ajout d'un projet

Test manuel dans Postman



Interface Postman – test de l'endpoint de récupération des projets

2. Précisez les moyens utilisés :

Langage utilisé :

- JavaScript avec utilisation de Node.js (24.10.0)

Framework utilisé

- Express

Packages utilisés

- bcryptjs pour le hachage des mots de passe
- jsonwebtoken (JWT) pour la génération et vérification des tokens d'authentification
- multer pour la gestion de l'upload des images
- express-validator pour la validation des données en entrée
- helmet et express-rate-limit pour la sécurisation de l'API contre les attaques courantes
- dotenv pour la gestion des variables d'environnement

Documentation et tests :

- Swagger pour la documentation interactive de l'API
- Postman pour les tests manuels des endpoints

Outils de développement :

- MongoDB Compass pour la visualisation des données

Éditeur de code :

- Visual Studio Code

Versionning

- Git Flow pour suivre et organiser les modifications
- GitHub pour héberger le code source

3. Avec qui avez-vous travaillé ?

DOSSIER PROFESSIONNEL (DP)

J'ai travaillé en autonomie sur ce projet en appliquant les compétences acquises durant la formation.

Ressources :

- Support de cours
- Documentation officielle Express
- Documentation officielle MongoDB
- Documentation officielle Mongoose
- Documentation officielle JWT
- Documentation officielle Swagger

4. Contexte

Nom de l'entreprise, organisme ou association ► **Global Digital University**

Chantier, atelier, service ► **Cours Node par la pratique**

Période d'exercice ► **Du 13/10/2025 au 20/10/2025**

5. Informations complémentaires *(facultatif)*

Activité-type 1 Développer une application sécurisée

CP 4 ► Contribuer à la gestion d'un projet informatique

1. Décrivez les tâches ou opérations que vous avez effectuées, et dans quelles conditions :

Dans le cadre de mon activité professionnelle en tant que développeuse depuis 6 ans, j'ai été amenée à participer activement à la gestion de projets informatiques, en collaboration étroite avec le chef de projet. Cette implication s'est concrétisée par la contribution à l'élaboration des spécifications fonctionnelles et techniques, l'estimation des charges, le suivi des tâches via des outils comme Monday et GitLab, ainsi que la gestion des demandes utilisateurs via GLPI dans le cadre de la Tierce Maintenance Applicative (TMA).

1 - Participation à l'élaboration des spécifications

Dans le cadre de mes missions en tant que développeuse, j'ai régulièrement contribué à l'élaboration des spécifications fonctionnelles et techniques en collaboration avec le chef de projet, les métiers et les utilisateurs finaux. Cette phase était cruciale pour cadrer les besoins et garantir que les développements répondent aux attentes.

Notre processus s'articulait autour des étapes suivantes :

- Recueil des besoins : Participation à des ateliers de cadrage avec les métiers pour identifier les fonctionnalités attendues.
- Rédaction des spécifications : Traduction des besoins en user stories avec des critères d'acceptation clairs.
- Validation : Relecture et ajustement des spécifications avec le chef de projet et les parties prenantes.
- Priorisation : Classement des fonctionnalités dans le backlog en fonction de leur valeur métier et de leur complexité technique, celle-ci pouvant être estimée grâce à des méthodes collaboratives comme le Planning Poker.

Exemple de backlog :

Pour illustrer cette contribution, je prends l'exemple du projet WebAbsence qui est un outil RH notamment de gestion d'absences. J'ai contribué à la création et à la maintenance d'un backlog structuré sous Excel, organisé en 5 onglets distincts, chacun répondant à un objectif précis dans le cycle de vie des spécifications et des développements.

Voici comment chaque onglet a été utilisé avec mon rôle dans leur gestion :

- Bac à sable :
Objectif : Collecter les idées brutes avant validation.
Mon rôle : Recueillir des besoins initiaux en participant à des ateliers avec les RH pour lister les besoins émergents. Rédaction de premières versions des user stories.
- Bac de culture :
Objectif : Affiner et détailler les user stories validées du *Bac à sable* pour les rendre prêtes à être développées.
Mon rôle : Rédaction des user stories détaillées avec des critères d'acceptation.

DOSSIER PROFESSIONNEL (DP)

- Backlog produit :
Objectif : Backlog officiel priorisé et prêt à être planifié en sprints.
Mon rôle : Classer les users stories avec le chef de projet en fonction de la valeur métier et des dépendances techniques. On ajoute une valeur d'importance et d'estimation pour chacune d'elle.

[illegible]

Backlog – Backlog de produit

- Bac de récolte :
Objectif : Stocker les user stories terminées (développées et validées).
Mon rôle : Déplacement des user stories validées en production vers ce bac.

	A	B	C	D	E	F	G	H
1	BACKLOG DE RÉCOLTE - WEBASBENCE					TOTAL VM PRODUITE : 24480		
2	ID	Catégorie	Nom	User Story	Importan	Estimatic	Démonstration	Commentaires
3	WEBA-66	Historique des absences	Écrire les absences	En tant que RH Je veux historiser des absences pour un collaborateur donnée Afin de contrôler et calculer	1200	150		
4	WEBA-67	Historique des absences	Calculer les droits	En tant que RH Je veux calculer les droits Afin de déterminer les soldes	1200	92		Cf Annexe 1 - Règles de gestion du calcul des droits
5	WEBA-52	Gestion RH	Ajouter un nouveau collaborateur	En tant que RH Je veux facilement ajouter un nouveau collaborateur	1020	78		Paramétrage du collaborateur (dates, compteurs, etc..)
6	WEBA-54	Gestion RH	Extraction des absences	En tant que RH Je veux extraire les absences à une période donnée	720	144		Extraire les données de toutes les personnes présentes à ces dates même si elles sont parties. Attention, conservation des données
7	WEBA-57	Gestion RH	Suppression Collaborateur	En tant que RH Je veux supprimer un collaborateur (visuellement uniquement)	420	210		Les informations doivent être conservées
8	WEBA-32	Visualisation du calendrier	Consulter les absences de mon pôle	En tant que utilisateur Je veux voir toutes les absences de mon pôle Afin d'avoir une vue d'ensemble des absences	1080	51		Planning sur lequel on peut sélectionner les collaborateurs à afficher
9	WEBA-68	Visualisation du calendrier	Consulter les absences de ma direction	En tant que utilisateur Je veux voir toutes les absences de mon pôle direction Afin d'avoir une vue d'ensemble des absences	1080	51		
10	WEBA-18	Gestion du calendrier	Ajouter mes prévisions d'absence	En tant que collaborateur Je veux ajouter mes prévisions d'absences	1140	54		Actions au clique / Demi-journée Actions possibles pour toutes les personnes de ma hiérarchie
	WEBA-20	Gestion du calendrier	Valider individuellement les demandes d'absence	En tant que valideur Je veux pouvoir valider individuellement	1140			Validation au type de congé

Backlog - Bac de récolte

DOSSIER PROFESSIONNEL (DP)

- Bac à glaçons
Objectif : Répertorier les user stories non prioritaires ou en attente de clarification.
Mon rôle : Identifier les blocages, contraintes techniques et questions relatifs à ces user stories.

2 - Estimation des temps et suivi de l'avancement

L'estimation des temps et le suivi de l'avancement permettent de planifier les projets et de garantir leurs livraisons dans les délais impartis.

Afin d'estimer les temps, je me base sur mon expertise et en faisant des analogies avec des fonctionnalités similaires déjà développées. Si une user story est trop complexe, on peut la décomposer en sous tâches pour aider à l'estimation.

J'ai participé également à des sessions de Planning Poker pour aider à l'estimation de la complexité des user stories. Ces sessions ont lieu avant la finalisation du backlog produit et en présence des développeurs et du chef de projet. Chaque user story est présentée et chaque personne doit lui attribuer un nombre de complexité (nombre de la suite de Fibonacci). Cela permet d'échanger autour du nombre que chacun a attribué et de déterminer une note de complexité à la user story qui fait consensus au sein de l'équipe.

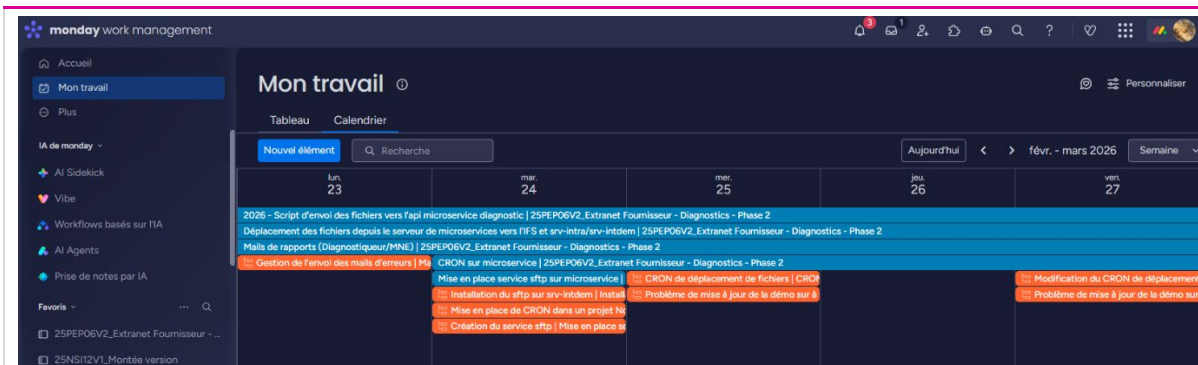
Lorsque les temps sont estimés, le chef de projet sélectionne des user stories et planifie des sprints. Les fonctionnalités sont ajoutées dans l'outil Monday avec l'estimation de temps de celles-ci et les délais de réalisations. Chaque développeur doit renseigner le temps passé sur ces fonctionnalités. Cela permet de visualiser facilement la progression des fonctionnalités et l'avancement du projet.



Élément	Personne	Statut d...	Suivi de la progre...	Échéances	Prévisionnel (jour...	Réalisé (jours)
Clôturer une prestation (dans le cas de rapports multiple)						
Clôturer une prestation en changeant le nombre de rap...		Fait	100%		0,5	0,25
Lister les prestations diagnostic						
1 Élément / 3 sous-éléments			100%		2	1,5
Contrôle des fichiers lors du dépôt						
1 Élément / 4 sous-éléments			0%	9 juin, '25 - 2...	3	1,625
Modification des rapports diagnostic						
Modification des rapports diagnostic		En cours	0%	9 juin, '25 - 2...	3	2,375

Interface Monday avec les fonctionnalités renseignées avec leurs échéances, les temps prévisionnels et les temps réalisés

DOSSIER PROFESSIONNEL (DP)



Interface Monday – vue calendrier avec les différentes tâches effectuées

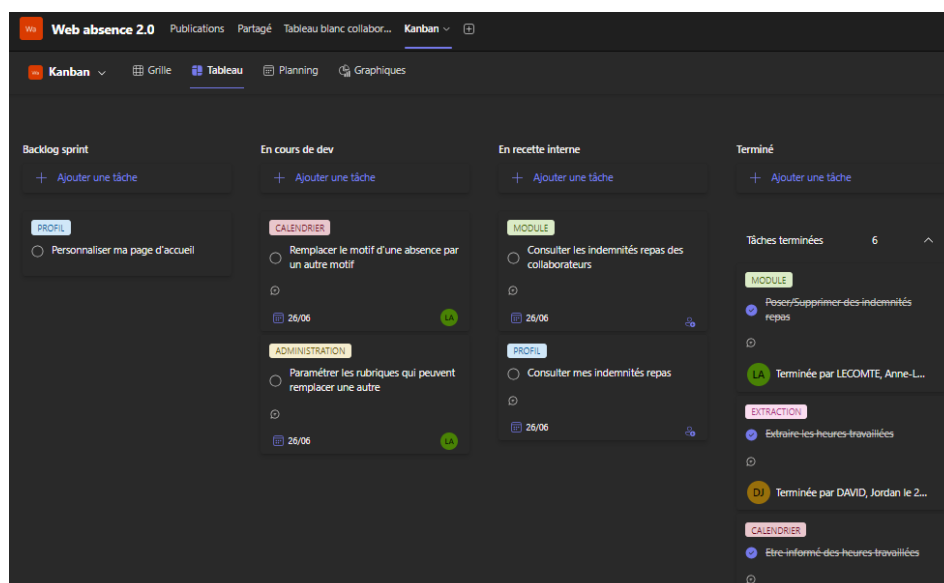
3 - Utilisation de la méthodologie Agile et du Kanban

Dans le cadre de certains projets, j'ai utilisé une méthodologie Agile avec des sprints de 3 semaines et en utilisant un tableau Kanban pour visualiser le flux de travail. Cela permet de livrer des fonctionnalités de manière régulière, de prioriser dynamiquement les tâches en fonction des urgences et d'être en collaboration étroite avec les métiers grâce à des rituels structurés.

Pour chaque sprint, des user stories sont sélectionnés en fonction de leurs priorisations. Celles-ci sont développées pendant deux semaines et demie. Ensuite on les déploie en environnement démonstration pour la Sprint Review où l'on fait une démonstration aux métiers des fonctionnalités développées afin qu'elles soient validées.

Nous utilisons un tableau Kanban dans l'application Teams où nous avons un canal dédié à chaque projet. Dans ce tableau, il y a 4 colonnes :

- Backlog sprint : User stories priorisées mais non encore planifiées en sprint
- En cours de dev : Tâches actuellement développées par l'équipe
- En recette interne : Tâches terminées mais non validées et en attente de test métier
- Terminé : Tâches validées et livrées en production



Teams – tableau Kanban

DOSSIER PROFESSIONNEL (DP)

4 - Gestion de la TMA

La TMA est une partie importante de la gestion de projet car elle permet d'assurer :

- La continuité du service pour les utilisateurs
- La résolution rapide des incidents
- L'évolution mineure de l'application

La gestion de la TMA se fait via deux outils :

- GLPI qui est un outil de gestion des tickets pour les demandes utilisateurs.
- GitLab qui est un outil de suivi technique pour classer, prioriser et résoudre les issues liées à la TMA des différents projets.

GLPI permet aux utilisateurs de faire des tickets lorsqu'ils rencontrent des problèmes ou qu'ils ont des demandes d'évolutions sur nos applications développées. Mon rôle est de faire un suivi des tickets qui me sont attribués. Pour cela j'analyse le ticket, vérifie la reproductibilité du problème et échange avec l'utilisateur.

GLPI

Accueil / Assistance / Tickets

Ajouter

Rechercher

Listes

Gabariets

Kanban global

Tickets attendant votre validation

Rechercher

Hotbar HTA12 44

Chercher dans le menu

Assistance

Tickets

Créer un ticket

Problèmes

Changements

Planning

Formulaire

Gestion

Outils

Configuration

Attribué à - Technicien

est

Moi-même

Régie

Régie globale

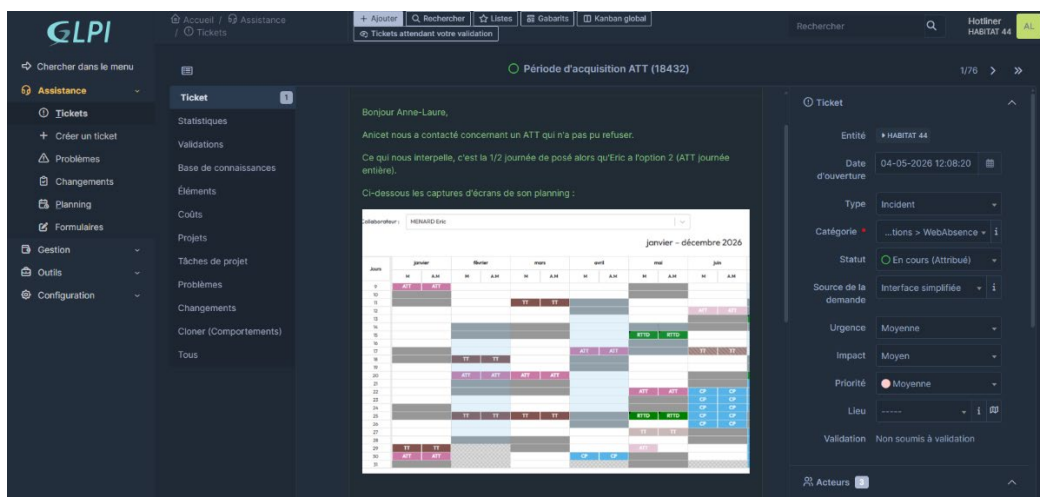
Hi groupe

Rechercher

Actions

ID	TITRE	PRIORITE	TYPE	DATE D'OUVERTURE	STATUT	DERNIERE MODIFICATION	DERNIER	DEMANDEUR	ATTRIBUE A - TECHNICIEN	TÂCHES	CATEGORIE
18 432	Période d'acquisition ATT	Moyenne	Incident	04-05-2026 12:08	En cours (Attribué)	07-05-2026 12:25	Carline LEMERLE	Carline LEMERLE	Anne-Laure LECOMTE	01 - Applications > WebAbsence	
18 361	Ajout destinataire mail intégration	Moyenne	Demande	20-04-2026 17:27	En cours (Attribué)	28-04-2026 16:45	Soleim HERMOULET	Soleim HERMOULET	Anne-Laure LECOMTE	01 - Applications > WebAbsence	
18 205	Intégration équipement DAACO dans IKOS	Moyenne	Demande	19-03-2026 09:48	Clos	24-04-2026 08:54	Ronan LABARRIERE	Caroline BOUYER	Le support informatique Justin MARAIS Anne-Laure LECOMTE Marion LAVENER	01 - Applications > IKOS > intervention BD	
18 163	accès serveur FTP	Moyenne	Incident	11-03-2026 15:19	En cours (Attribué)	21-04-2026 11:06	Justin MARAIS	Sophie LEFREVOST	Justin MARAIS Anne-Laure LECOMTE	01 - Applications > Outils I44	
18 268	Blocage WA	Haute	Incident	31-03-2026 09:05	Clos	21-04-2026 08:44	Anne-Laure LECOMTE	Carline LEMERLE	Anne-Laure LECOMTE	01 - Applications > WebAbsence	
18 242	Bailson webabsences/outlook	Moyenne	Incident	25-03-2026 09:08	Clos	16-04-2026 08:37	Anne-Laure LECOMTE	Amandine CORBINEAU	Anne-Laure LECOMTE	01 - Applications > WebAbsence	
18 309	Ajustement module délégation	Moyenne	Demande	08-04-2026 16:54	En cours (Attribué)	09-04-2026 10:41	Soleim HERMOULET	Soleim HERMOULET	Anne-Laure LECOMTE	01 - Applications > WebAbsence	
18 251	ACCES SERVEUR STP	Moyenne	Demande	26-03-2026 14:23	En cours (Attribué)	27-03-2026 11:28	Justin MARAIS	Fabien RAPIN	Justin MARAIS Anne-Laure LECOMTE	01 - Applications > Extranet Fournisseur	

Interface GLPI – liste des tickets



Entité	Date d'ouverture	Type	Catégorie	Statut	Source de la demande	Urgence	Impact	Priorité	Validation
HABITAT 44	04-05-2026 12:08:20	Incident	Applications > WebAbsence	En cours (Attribué)	Interface simplifiée	Moyenne	Moyen	Moyenne	Non soumis à validation

Interface GLPI – vue d'un ticket

DOSSIER PROFESSIONNEL (DP)

Je fais le lien avec GitLab où je vais créer une issue sur le projet en cause.
Dans le Gitlab, nous avons mis en place une qualification des issues avec un diagramme permettant de nous aider à les qualifier.

Prioritized Labels			
Bloquant	extreme-fournisseurs / front / extreme-fournisseur-v2	Issues - Merge requests · Prioritized label	Subscribe
Other Labels			
Amélioration	Amélioration liée au projet qui n'a pas été définie dans les spécifications en amont extreme-fournisseurs / front / extreme-fournisseur-v2	Issues - Merge requests	Subscribe
Boîte à idée	Proposition de l'équipe dev extreme-fournisseurs / front / extreme-fournisseur-v2	Issues - Merge requests	Subscribe
Correctif	Correctif suite à une problématique rendant le projet inaccessible OU provoquant un message d'erreur OU ne correspondant pas au comportement souhaité défini dans les spécifications extreme-fournisseurs / front / extreme-fournisseur-v2	Issues - Merge requests	Subscribe
Crucial	Correctif/Amélioration essentiel au bon fonctionnement de l'application extreme-fournisseurs / front / extreme-fournisseur-v2	Issues - Merge requests	Subscribe
En attente	extreme-fournisseurs / front / extreme-fournisseur-v2	Issues - Merge requests	Subscribe
Evolution	Nouvelle fonctionnalité demandée par un utilisateur extreme-fournisseurs / front / extreme-fournisseur-v2	Issues - Merge requests	Subscribe
Majeur	Correctif/Amélioration suite à une problématique contraignante dans l'utilisation quotidienne de l'outil OU évolution essentielle au bon fonctionnement de l'application extreme-fournisseurs / front / extreme-fournisseur-v2	Issues - Merge requests	Subscribe
Mineur	Correctif/Amélioration suite à une problématique non-contraignante dans l'utilisation quotidienne de l'outil OU évolution non-essentielle au bon fonctionnement de l'outil extreme-fournisseurs / front / extreme-fournisseur-v2	Issues - Merge requests	Subscribe

Les différentes qualifications des issues

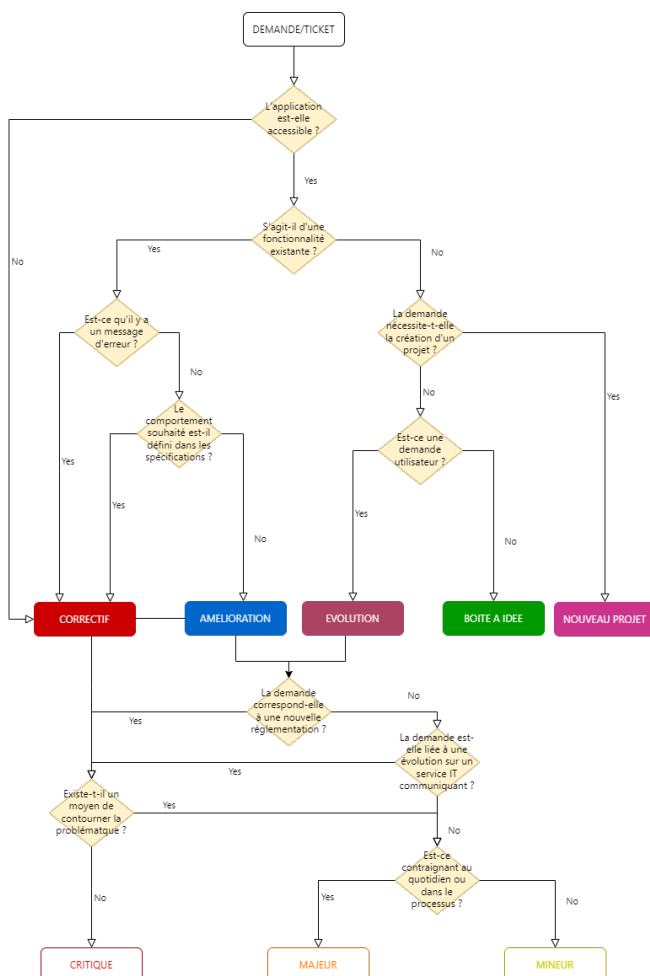
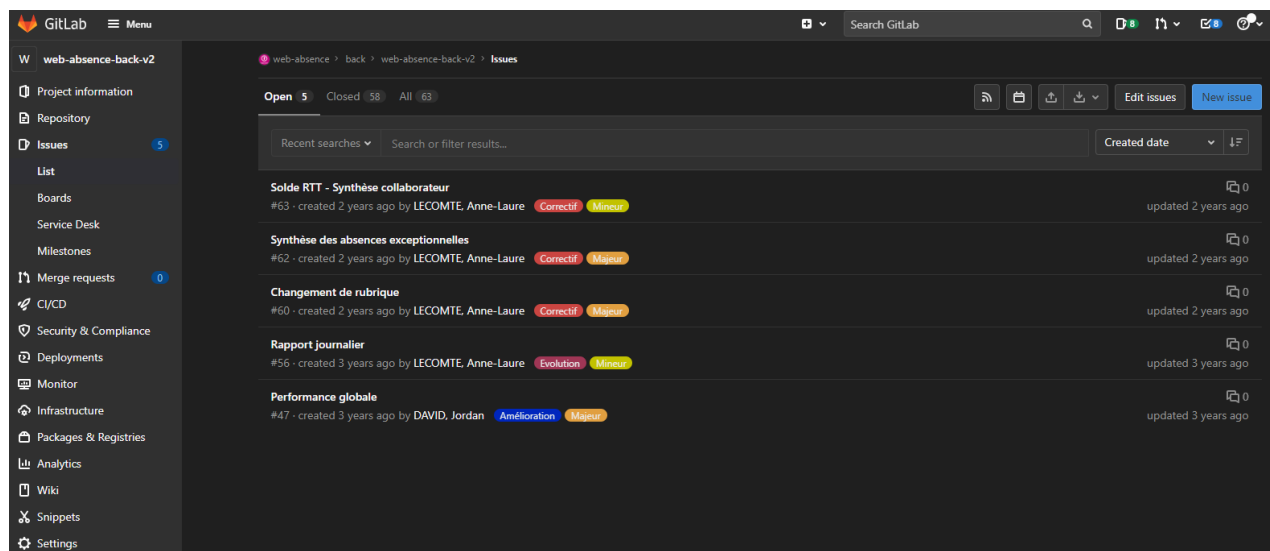


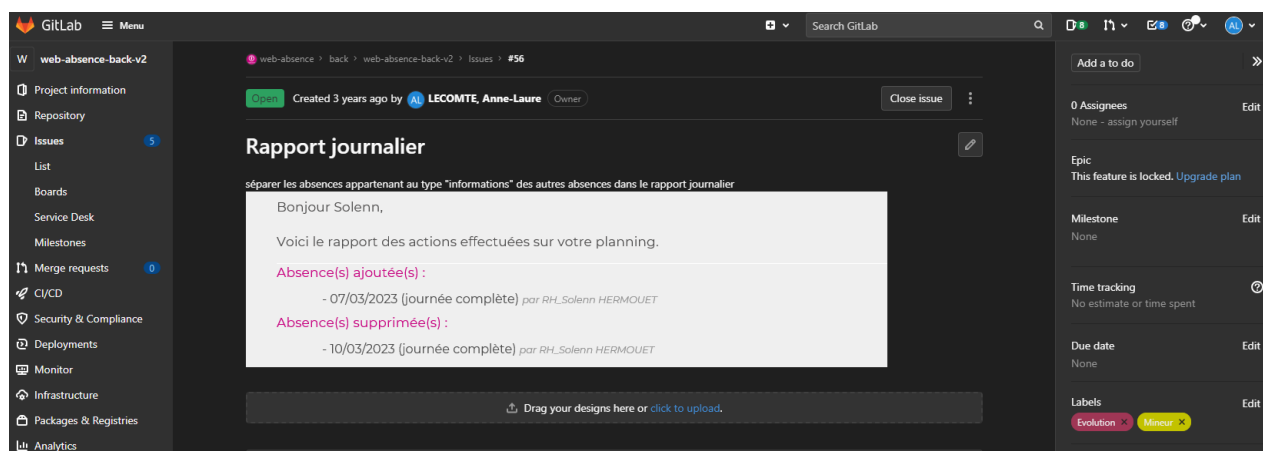
Diagramme de qualification

DOSSIER PROFESSIONNEL (DP)

En fonction de cette qualification, nous allons créer une nouvelle branche sur le projet en utilisant GitFlow. Cela facilite la gestion des versions, leurs déploiements et la collaboration au sein de l'équipe grâce aux règles de branchement définies.



Interface GitLab – liste des issues liées à un projet



Interface GitLab – vue d'un ticket

Ce fonctionnement permet une bonne réactivité sur les tickets critiques via le déploiement de hotfix et permet d'avoir une traçabilité sur les problèmes des applications et les demandes utilisateur.

La gestion de la TMA permet d'assurer un support réactif et une amélioration continue des applications. En l'associant à la méthodologie Agile et au suivi des temps, cette approche contribue à mettre en place une gestion de projet équilibrée entre rigueur technique et adaptabilité où les développeurs travaillent en étroite collaboration avec le chef de projet et les métiers.

DOSSIER PROFESSIONNEL (DP)

2. Précisez les moyens utilisés :

Outils utilisés :

- Monday
- Teams
- GLPI
- GitLab
- GitFlow
- Excel

3. Avec qui avez-vous travaillé ?

J'ai travaillé en collaboration le chef de projet, d'autres développeurs, les métiers et les utilisateurs finaux dans le cadre de réunions pour l'élaboration des spécifications et de présentations d'avancement de projet.

4. Contexte

Nom de l'entreprise, organisme ou association ► **Habitat 44**

Chantier, atelier, service ► **Systèmes d'informations**

Période d'exercice ► **Du 16/03/2020 au 30/04/2026**

5. Informations complémentaires (facultatif)

DOSSIER PROFESSIONNEL (DP)

DOSSIER PROFESSIONNEL (DP)

Titres, diplômes, CQP, attestations de formation

(facultatif)

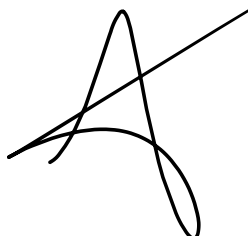
Intitulé	Autorité ou organisme	Date
Titre professionnel Développeur web et web mobile	Ministère du travail	15/06/2020
BA in Graphic Arts : Photography and Digital Media	University of Southampton	22/06/2012
BTS Communication Visuelle option « graphisme, édition, publicité »	Ministère de l'enseignement supérieur et de la recherche (Académie de Nantes)	23/10/2010
HND in Graphic Design	Edexcel	18/07/2010
Cliquez ici.	Cliquez ici pour taper du texte.	Cliquez ici pour sélectionner une date.
Cliquez ici.	Cliquez ici pour taper du texte.	Cliquez ici pour sélectionner une date.
Cliquez ici.	Cliquez ici pour taper du texte.	Cliquez ici pour sélectionner une date.
Cliquez ici.	Cliquez ici pour taper du texte.	Cliquez ici pour sélectionner une date.
Cliquez ici.	Cliquez ici pour taper du texte.	Cliquez ici pour sélectionner une date.
Cliquez ici.	Cliquez ici pour taper du texte.	Cliquez ici pour sélectionner une date.

Déclaration sur l'honneur

Je soussigné(e)**Anne-Laure Lecomte**..... ,
déclare sur l'honneur que les renseignements fournis dans ce dossier sont exacts et que je
suis l'auteur(e) des réalisations jointes.

Fait à *Nantes*..... le *08/05/2026*.....
pour faire valoir ce que de droit.

Signature :



DOSSIER PROFESSIONNEL (DP)

Documents illustrant la pratique professionnelle

(facultatif)

Intitulé
Cliquez ici pour taper du texte.

DOSSIER PROFESSIONNEL (DP)

ANNEXES

(Si le RC le prévoit)

